



Machine Learning for Liquid Rocket Engine Startup

Prepared by

ATA Engineering, Inc.
13290 Evening Creek Drive South
San Diego, California 92128

Signatures and Contact Information

Prepared by:

Aiden Kampwerth

Project Manager

Aerospace Engineer Intern

Reviewed by:

Victoria Harris

Reviewer

AIML Technical Director

CONTACTS

ATA:

Aiden Kampwerth 720.456.4170

aiden.kampwerth@ata-e.com

Emma Joly 720.456.4152

emma.joly@ata-e.com

Victoria Harris 703.225.7447

victoria.harris@ata-e.com

ABSTRACT

This report documents ATA Engineering, Inc.'s current and future work on Intelligent Gradient-based Neural-network for Ignition Timing Evaluation (IGNITE). IGNITE is a proof-of-concept software package aimed at predicting the open-loop ignition sequence for a liquid rocket engine (LRE) utilizing a reinforcement learning (RL) model and Siemen's Amesim software. The RL agent predicts a stable ignition sequence utilizing Amesim as an environment from which it can learn, explore, and exploit properties of LREs at least as fast as a field expert. Current work involves testing the RL agent against simplified environments for verification and validation with future work entailing the application of the RL agent in the LRE environment and implementing significant performance improvements.

DOCUMENT REVISION LOG

Version	Description	Author	Date
1.0	Initial release	A. Kampwerth / E. Joly	5/3/2024
1.1	Rev 1	A. Kampwerth / E. Joly / V. Harris	5/3/2024

TABLE OF CONTENTS

1. INTRODUCTION	1
1.1. Background.....	1
1.2. Problem Statement	2
1.3. Applications.....	3
2. SIMULATION ENVIRONMENT.....	5
2.1. Amesim Platform	5
2.1.1. Components	5
2.1.2. Libraries	6
2.1.3. Variables	7
2.2. LRE Controller	8
2.2.1. RS-25 Valves	9
2.2.2. RS-25 Reference Ignition Sequence.....	10
2.2.3. Open Loop vs. Closed Loop Control	11
2.3. Functional Mock-up Interface.....	11
2.3.1. Functional Mock-up Unit.....	11
2.3.2. FMU Model Types.....	11
2.3.3. FMI Integration in Amesim.....	13
3. REINFORCEMENT LEARNING AGENT.....	15
3.1. Soft Actor-Critic Architecture.....	15
3.1.1. Actor Network	16
3.1.2. Critic and Target Critic Network.....	17
3.1.3. Entropy Parameter	17
3.2. Training Structure.....	18
3.2.1. Critic Training.....	19
3.2.2. Actor Training.....	21
3.2.3. Entropy Training	22
3.2.4. Target Critic Update	22
3.3. Reward Structure.....	23
3.3.1. Prime Checks.....	24
3.3.2. Transient Rewards	24
3.3.3. Ignition Rewards.....	24
3.3.4. Violations	25

3.4.	Agent Hyper-Parameters	26
3.4.1.	Learning Rate	26
3.4.2.	Entropy Scale.....	26
3.4.3.	Training Threshold.....	26
3.4.4.	Network Layer Size and Depth	26
4.	IGNITE STRUCTURE.....	27
4.1.	Variable Handling	27
4.2.	Main Loop	28
4.3.	Episode Loop	29
4.4.	GUI	30
4.4.1.	Input Window.....	30
4.4.2.	Log Window	31
4.4.3.	Plot Window	31
4.5.	Post-Processing	32
5.	VERIFICATION AND VALIDATION.....	33
5.1.	Environment.....	33
5.1.1.	FMU Variables	33
5.1.2.	General FMU Simulation.....	34
5.1.3.	Reference Ignition Sequence Simulation	34
6.	RESULTS AND ANALYSIS	35
6.1.	Toy Problems.....	35
6.1.1.	Linear State Actions	35
7.	FUTURE WORK	43
7.1.	LRE Stability Check.....	43
7.2.	Updated Reward Structure	43
7.3.	Clustered Environment Simulation.....	44
7.4.	Agent Pre-Training	44
	REFERENCES.....	45

LIST OF FIGURES

Figure 1-1. RS-25 ignition sequence [10].	2
Figure 1-2. Starship exploding due to several Raptor engine failures [15].	3
Figure 2-1. naval ship and engine modeled in Amesim [20].	5
Figure 2-2. Liquid Propulsion libraries' nozzle component port diagram.	6
Figure 2-3. RS-25 oxygen pre-burner Amesim model.	7
Figure 2-4. combustion chamber state variables after a simulation.	8
Figure 2-5. RS-25 engine controller in Amesim environment.	8
Figure 2-6. SSME reference ignition sequence [16].	10
Figure 2-7. Model Exchange diagram [17].	12
Figure 2-8. Co-Simulation diagram [17].	12
Figure 2-9. Amesim FMI interface component.	13
Figure 2-10. RS-25's engine controller.	14
Figure 3-1. generic RL diagram [18].	15
Figure 3-2. Soft Actor-Critic overview diagram [19].	16
Figure 3-3. Soft Actor-Critic overall training diagram.	18
Figure 3-4. critic training diagram.	19
Equation 3-1. simplified Bellman's Equation.	20
Figure 3-5. actor training diagram.	21
Figure 3-6. entropy training diagram.	22
Figure 3-7. target critic update diagram.	22
Figure 3-8. target value reward function.	23
Figure 3-9. target delay reward function.	25
Figure 3-10. violation penalty function.	25
Figure 4-1. StateVariable class structure and members.	27
Figure 4-2. main loop diagram.	29
Figure 4-3. episode loop diagram.	29
Figure 4-4. GUI main window.	30
Figure 4-5. input section of main window in the GUI.	31
Figure 4-6. log section of main window in the GUI.	31
Figure 4-7. post-processing window.	32
Figure 4-8. plot section of the main window.	32
Figure 5-1. example of FMU variables.	33
Figure 6-1.1. static entropy and LR reward history.	35
Figure 6-1.2. static entropy and LR final ignition sequence.	36

Figure 6-2.1. dynamic entropy reward history.....	37
Figure 6-2.2. dynamic entropy final ignition sequence.	38
Figure 6-2.3. dynamic entropy actor loss.	38
Figure 6-2.4. dynamic entropy critic loss.....	39
Figure 6-2.5. dynamic entropy entropy term.....	39
Figure 6-3.1. dynamic entropy and LR reward history.....	40
Figure 6-3.2. dynamic entropy and LR final ignition sequence.....	40
Figure 6-3.3. dynamic entropy and LR actor loss.	41
Figure 6-3.4. dynamic entropy and LR critic loss.....	41
Figure 6-3.5. dynamic entropy and LR entropy term.	42

1. INTRODUCTION

When developing Liquid Rocket Engines (LREs), an often-overlooked problem is the development of an open-loop control ignition sequence to transition an LRE from off to full throttle. This project aims to create a machine learning model capable of predicting an ignition sequence to augment the work of propulsion engineering experts which could save time, money, and potentially lives.

An aside, readers working on this project are heavily encouraged to review the training material in Appendix A. Basic terminology will not always have an explanation or reference example and uninformed readers may miss crucial details.

1.1. Background

LREs are not complex due to the number of components, but rather due to the strong inter-dependence of the few components there are. LREs must contain all of their subsystems onboard so utilizing multiple independent subsystems is both a waste of space and mass. Therefore, an intricately connected LRE is not only preferred but often necessary to achieve performance and efficiency metrics.

However, the inherent complexity of an LRE raises several challenges, with the primary concern being control. LREs experience some of the most extreme conditions known within sometimes only a few centimeters of each other. It is imperative that these conditions are managed correctly or [rapid unscheduled disassemblies](#) will occur.

Typically, control becomes trivial in steady state conditions, such as reaching a target Rated Power Level (RPL). Issues arise during the transient phase between steady state conditions where several components need to make significant changes to their internal conditions at the same time [11]. The relatively few control points (fluid valves) need to gradually move the internal states of components towards their target without exceeding numerous physical limits of the engine.

The primary occurrence of this transition is during LRE ignition right before launch. Failure to properly control the internal LRE state can lead to events like [unstarts](#) where the combustion propagates upstream through the delicate components instead of being exhausted through the nozzle.

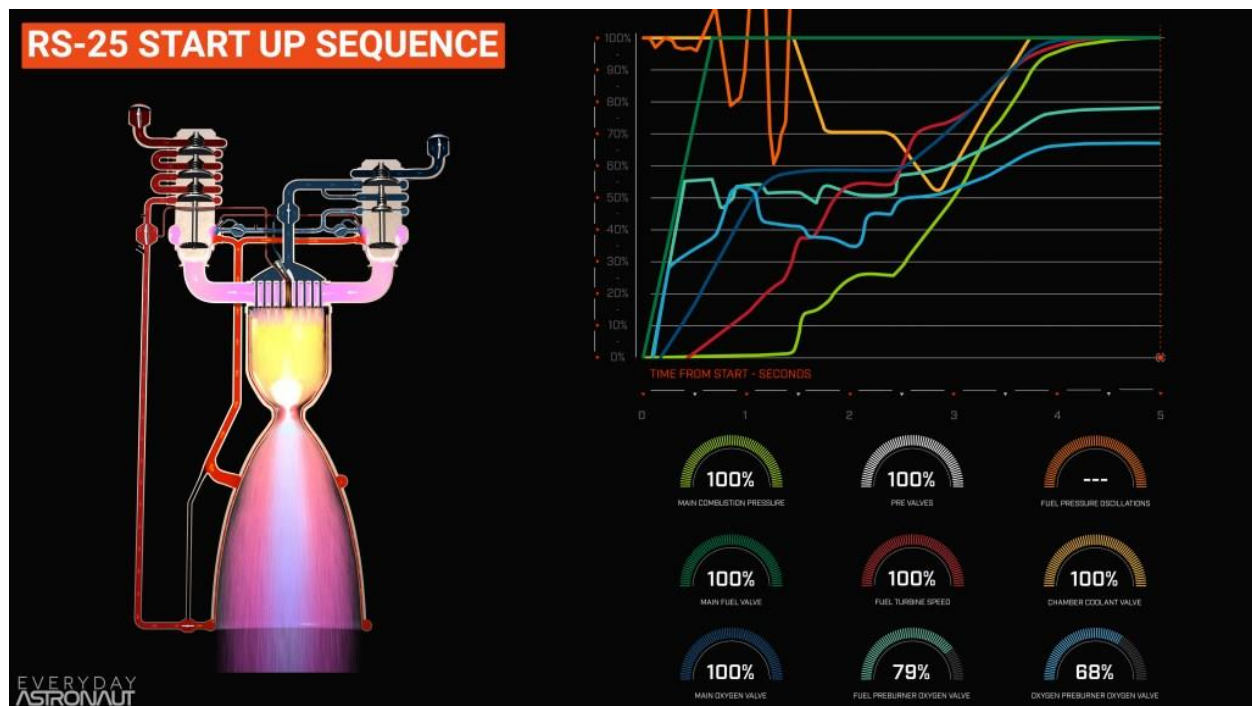


Figure 1-1. RS-25 ignition sequence [10].

Given the inexact understanding of a LRE's transient state properties during the development phase, propulsion engineers often have to manually control the internal state to prevent failures. This is accomplished via an ignition sequence, a timing graph which dictates the position of fluid valves over the course of the ignition sequence as seen in Figure 1-1.

Even today, the only completely accurate way of determining the ignition sequence is through testing prototypes until one doesn't explode. The RS-25's development include over 50 static fire tests before it stopped exploding during ignition [13]. Propulsion engineers try to reduce the required number of static fires by using numerous calculations, simulation models, and often decades of field experience to determine a guess for what the ignition sequence should roughly be. This guess will be referred to as an initial ignition sequence for this project.

The required work to get an initial ignition sequence has barely decreased since the development of the RS-25 in the 1970's. Modern simulation software like Siemens's [Amesim](#) (discussed further in section 2.1) has replaced NASA's outdated [ROCETS](#), but little progress has been made to truly reduce the required work.

1.2. Problem Statement

Therefore, this project aims to generate an initial ignition sequence for any LRE utilizing only its design parameters and simulation model.

For a solution to this problem, the following requirements must be met:

- Generalizability
- Precise control
- No additional information or analysis
- At least as fast as an expert

These requirements led to the use of a Reinforcement Learning (RL) agent, capable of learning any LRE through trial and error and only given the simulation model with a reward structure (discussed further in section 3).

The RL agent combined with the Amesim environment is wrapped into the Intelligent Gradient-based Neural-network Ignition Timing Evaluation (IGNITE) software.

As for the specific LRE, the RS-25 was chosen due to being a pre-built Amesim model and the numerous publicly available documents regarding its development. This is not typical for LREs as their information is usually proprietary and therefore inaccessible. Even given the open nature of the RS-25, much of the raw data is kept with NASA's subcontractors, hence the need for machine learning methods which generate their own data.

1.3. Applications

For the majority of LREs, their complexity directly correlates to a more difficult ignition sequence. The most performant LREs, such as a [full-flow staged combustion cycles](#) (FFSC), have historically been exceptionally difficult to control properly. Currently, SpaceX has been the only organization to develop a FFSC LRE verified for orbital flight ([Raptor](#)) partly due to the ignition sequence they found. Therefore, IGNITE would enable other organizations to develop their own FFSC LREs much faster and directly compete with SpaceX.

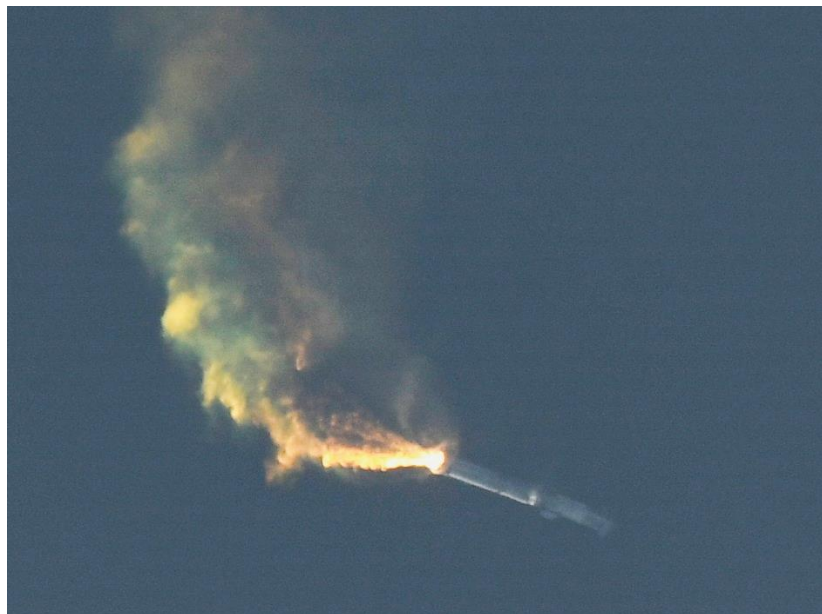


Figure 1-2. Starship exploding due to several Raptor engine failures [15].

A separate issue when designing LREs are determining failure modes. Only after an ignition sequence is found can propulsion engineers accurately predict failure modes and improve the design [8]. However, this is often an afterthought as a majority of the failure modes are discovered trying to find the ignition sequence. For example, SpaceX's Integrated Flight Test (IFT) 1 had several separate LRE failure events occur which resulted in the termination of the flight.

IGNITE would allow for a shift to determine failure modes before any prototypes need to be created. Designers can then use the results they gathered to implement or modify features in the LRE design to prevent disasters like in Figure 1-2.

2. SIMULATION ENVIRONMENT

The RL agent requires an environment to work in which can be accessed through Python. This section addresses the following:

- The specifics of the simulation environment, Siemens' Amesim
- Representation of the RS-25 in of the simulation environment
- How the simulation environment can be imported into the main program

2.1. Amesim Platform

Siemens' Amesim is a simulation software platform developed primarily for use in modeling and analyzing multi-domain systems. It allows engineers to perform simulations of mechanical, hydraulic, pneumatic, thermal, and electrical systems within a single integrated environment. The platform is particularly valuable for its ability to help predict the behavior of the systems and optimize their performance before actual physical prototypes are built.

The optimization and prediction capabilities are the inspiration for this project and are used in a variety of ways to achieve our end goal.

2.1.1. Components

Amesim's primary building blocks are components which each model a specific subsystem within an overall system.

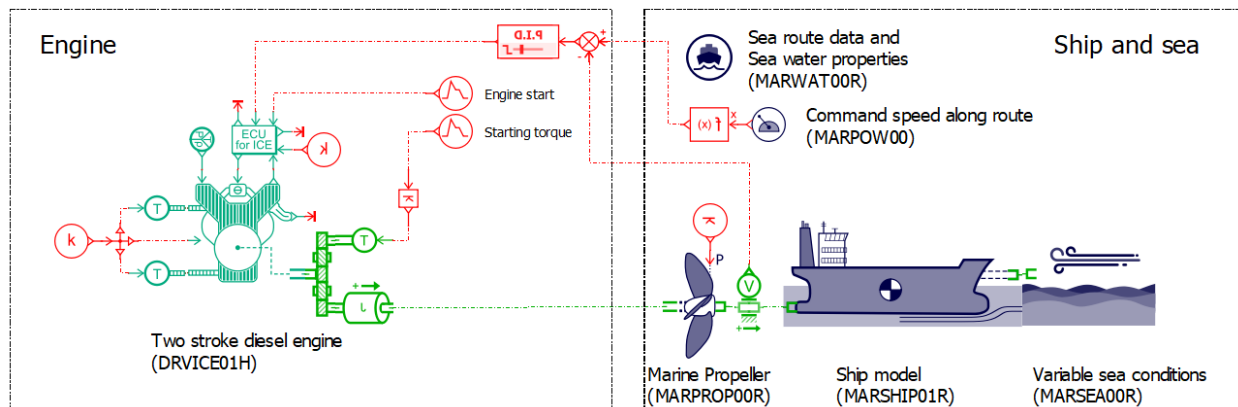


Figure 2-1. naval ship and engine modeled in Amesim [20].

For example, in Figure 2-1, the ship's diesel engine is represented with multiple rotary components (colored in green), fluid-mechanical components (colored in turquoise), and wiring components (colored in red).

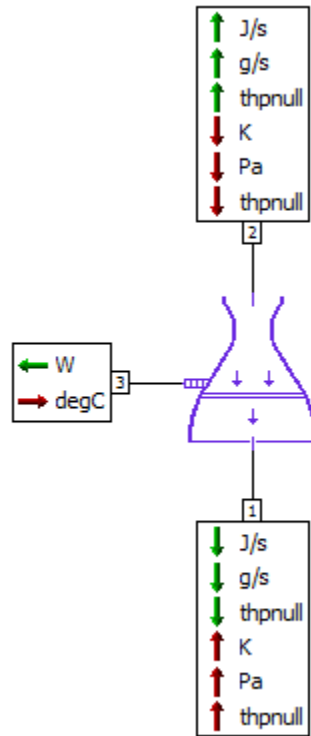


Figure 2-2. Liquid Propulsion libraries' nozzle component port diagram.

All of the components have input-output ports which allows information to flow across the model. These are represented by the red and green arrows in Figure 2-2. All component ports in a model must be connected in order to properly transmit information and create a closed system.

Some components are general representations of a subsystem and therefore a specific submodel must be assigned to capture the appropriate dynamics of the system. This also can simplify certain models by combining multiple components for a subsystem into a single general component with a submodel.

2.1.2. Libraries

Amesim categorizes its components into libraries which each serve a general domain. For this project, the Liquid Propulsion, Thermal, and Two-Phase Flow libraries provided the majority of components required to model the RS-25.

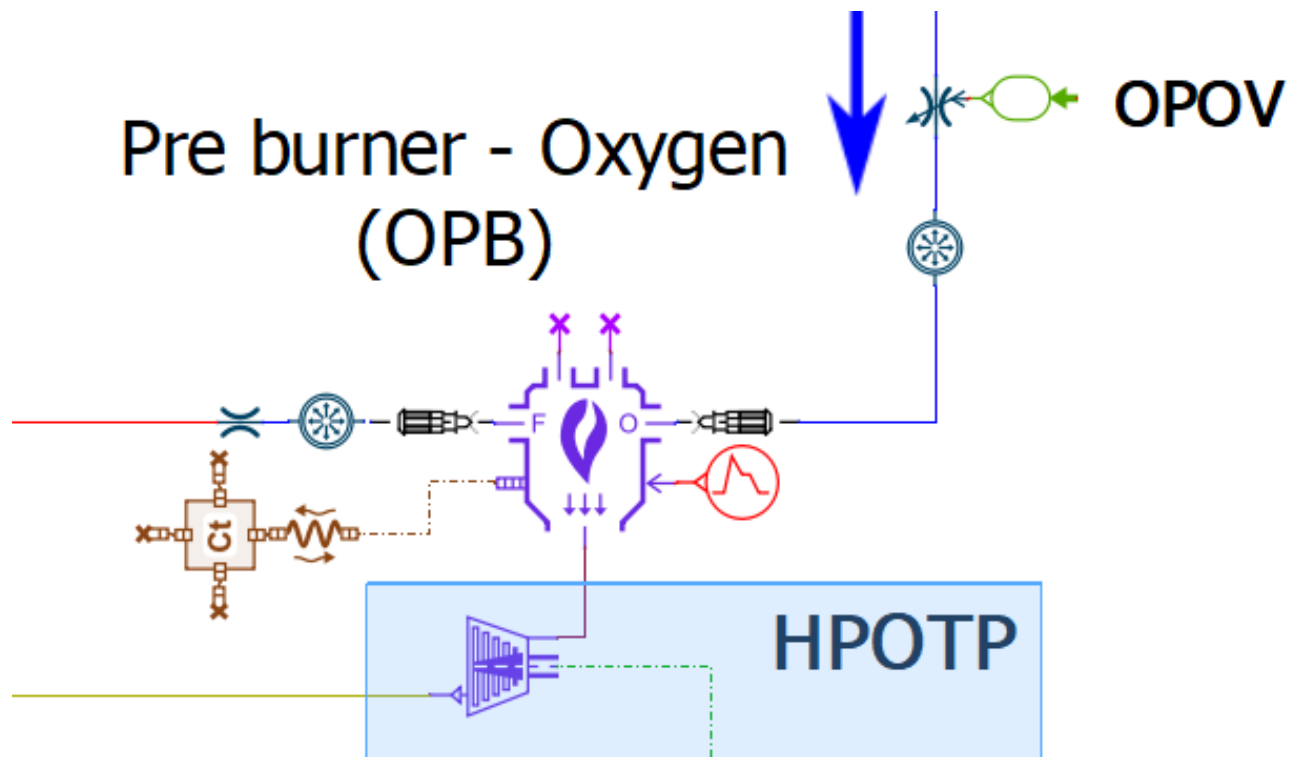


Figure 2-3. RS-25 oxygen pre-burner Amesim model.

Two-Phase Flow provides all of the components that directly handle the transportation and manipulation of fluids throughout the LRE. (red/blue/yellow/brown lines connecting components)

Liquid Propulsion contains the major LRE components such as the combustion chambers, nozzles, turbines, etc. (colored in purple)

Thermal provides components responsible for thermal management. (colored in brown)

2.1.3. Variables

The primary way Amesim models the interaction of components is through state variables (refer to Figure 2-2). Each component has state variables which are governed by Ordinary Differential Equations (ODEs) specific to the component. Therefore, when state variables are coupled together in a model, a system of ODEs is formed which an ODE integrator can solve numerically.

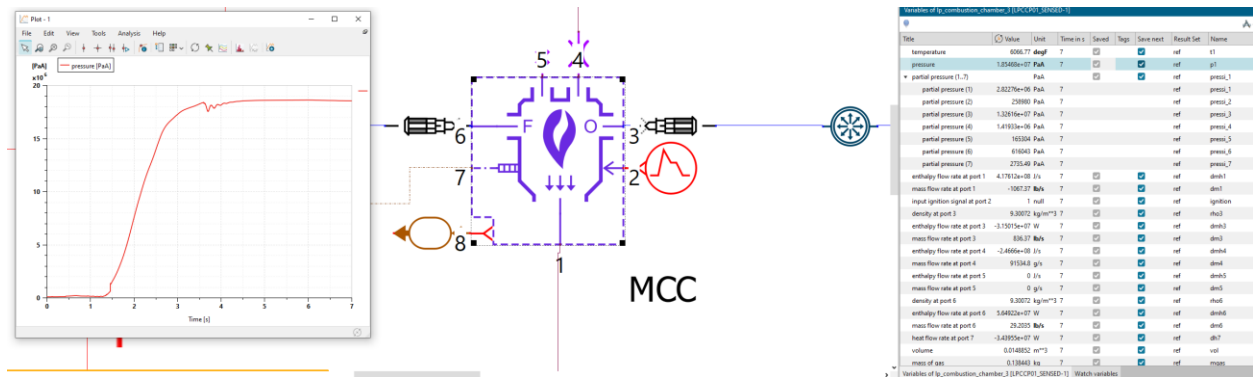


Figure 2-4. combustion chamber state variables after a simulation.

Due to the numerical solution, the solver steps through time instead of generating the entire solution. This enables the agent to interact with the environment and receive feedback in the form of updates to the state variables.

2.2. LRE Controller

The RS-25 has 5 primary valves used for the majority of the control in the LRE [10]. They are as follows:

- Main Oxidizer Valve (MOV)
- Main Fuel Valve (MFV)
- Oxidizer Pre-Burner Oxidizer Valve (OPOV)
- Fuel Pre-Burner Oxidizer Valve (FPOV)
- Coolant Chamber Valve (CCV)

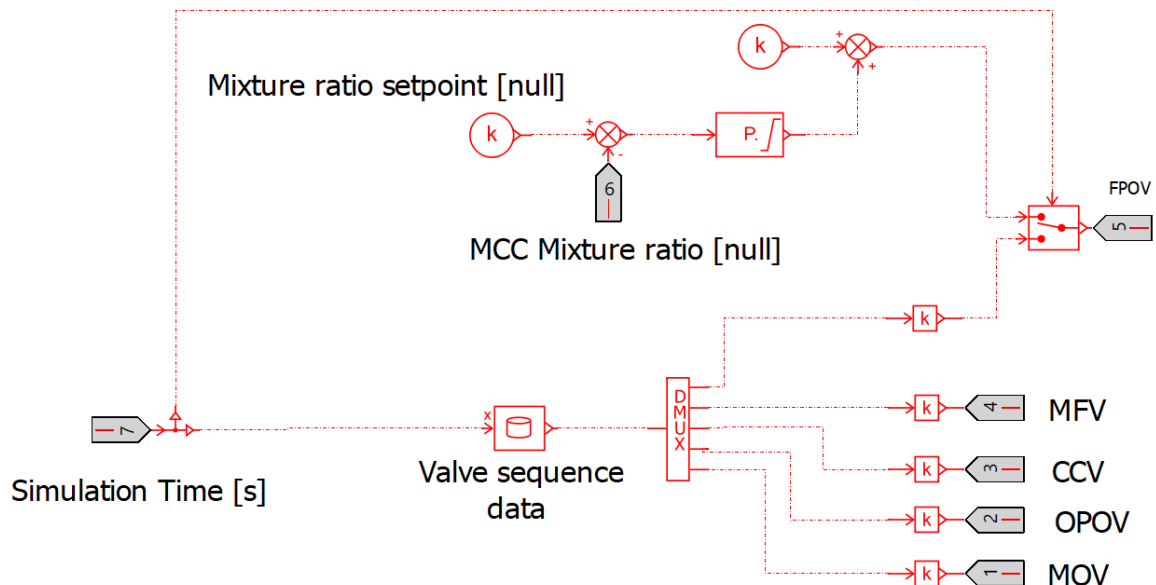


Figure 2-5. RS-25 engine controller in Amesim environment.

Figure 2-5 depicts how all 5 valves are integrated within the Amesim environment. The goal of the project is to replace the “valve sequence data” container with an agent capable of controlling all the valves at each instance.

The following sections analyze the macro properties of the RS-25 with relation to the valve control to improve the performance of the agent.

2.2.1. RS-25 Valves

The MOV and MFV control the total flow of oxidizer and fuel into the entire LRE, respectively. The RS-25 uses a staged combustion cycle with two fuel rich pre-burners which means all the fuel runs through the pre-burners while only part of the oxidizer does.

Therefore, the missing pre-burners’ fuel valves are accounted for by the MFV. Consequently, the inherent lack of control over the fuel in the LRE forced the designers to set the position of the MFV as a design parameter. This eliminates the need for the agent to model the MFV entirely.

Furthermore, the oxidizer flow into the pre-burners is controlled by the OPOV and FPOV which are fed by the MOV. The MOV can then be modeled to be the sum of the required mass flow rates (MFR) of the pre-burners and the requested MFR of the main combustion chamber (MCC). Therefore, the MOV can be directly associated with control of the MCC’s oxidizer.

As an aside, the CCV only re-directs a part of the fuel flow towards the LRE’s nozzle walls to cool the engine down. This fuel is then reintroduced to the main flow after passing around the nozzle with the only major difference between the two being the temperature of the fuel. The added rise in temperature of the fuel is considered a design parameter and its control is only associated with maintaining a specific temperature in the nozzle.

The pre-burners are directly associated with several major components in the LRE and thus their internal state is particularly important to LRE stability. Given the other three valves are considered design parameters, the OPOV and FPOV are the primary control points for the RS-25.

Combining these notes together enables the creation of relevant reward systems in the agent without compromising generalizability of IGNITE.

2.2.2. RS-25 Reference Ignition Sequence

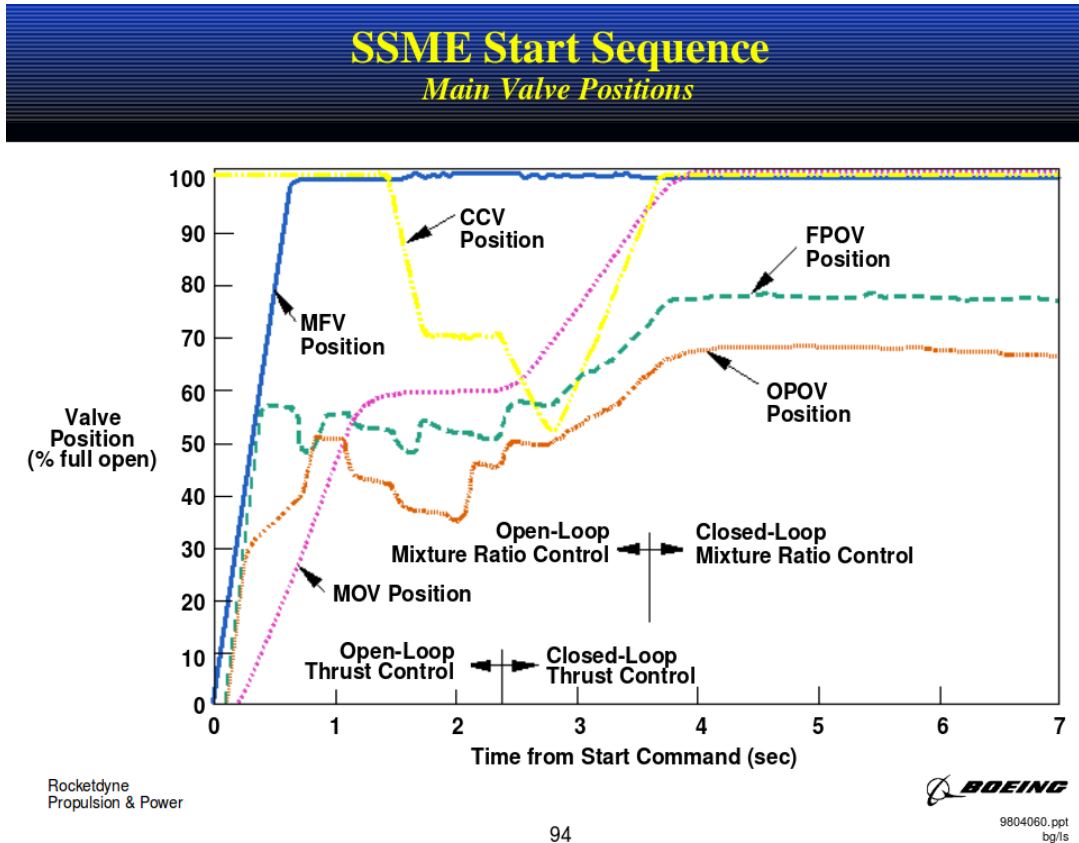


Figure 2-6. SSME reference ignition sequence [16].

Figure 2-6 displays the 5 primary valves' position over the duration of the ignition sequence.

The MFV, MOV, and CCV follow a particular pattern of moving towards a position and holding at that position for some period. This is due to these valves being design parameters and therefore only adjusting to major changes in the internal state.

The OPOV and FPOV are instead highly variable and are used to account for minor changes in the internal state. They still follow the pattern of the 3 previous valves; however, they operate in a region around the target position instead of a fixed value.

The two separate behaviors are classified as fixed and variable valves which can be applied to LREs regardless of number of valves or complexity. Modifying the agent to exploit these behaviors can significantly reduce training time compared to modeling each valve across the entire domain.

2.2.3. Open Loop vs. Closed Loop Control

The goal is to model only the open-loop part of the ignition sequence as a closed-loop region is not directly controlled by the designers. The RS-25 has two closed-loop regions: Thrust Control and Mixture Ratio Control.

The mixture ratio control region watches the mixture ratio (MR) in the MCC and adjusts the FPOV valve accordingly to keep a MR of 7. The RS-25's computer has little to no added input in this region and the LRE is in a steady state (SS) condition.

The thrust control region is more nuanced and only occurs after specific control laws become applicable. The designers of the RS-25 tested the LRE numerous times before being able to accurately control the LRE in this region solely based on computer commands. Therefore, the final software package should include the ability to model this region, however, the IRAD will not model the thrust control region for simplicity's sake.

In summary, the agent will only model the valve positions until a target internal condition is reached. This internal state will function as an overall stability criterion in addition to aiding in the reward structure for the agent.

2.3. Functional Mock-up Interface

While Amesim has an internal python console, accessing it and manipulating state variables proved to be difficult and unnecessary. Amesim provides a more user-friendly approach through the Functional Mock-up Interface ([FMI](#)). FMI is an industry standard format for simulation models which facilitates inter-software compatibility.

2.3.1. Functional Mock-up Unit

FMI models are defined as Functional Mock-up Units (FMUs) which are a combination of C code and XML files. The C code contains the system of ODEs, state variables, inputs, and outputs from the original simulation platform. The XML files provide a description of the FMU and other, non-relevant secondary functions.

2.3.2. FMU Model Types

A majority of FMUs can be split into Co-Simulation (CS) and Model Exchange (ME) model types.

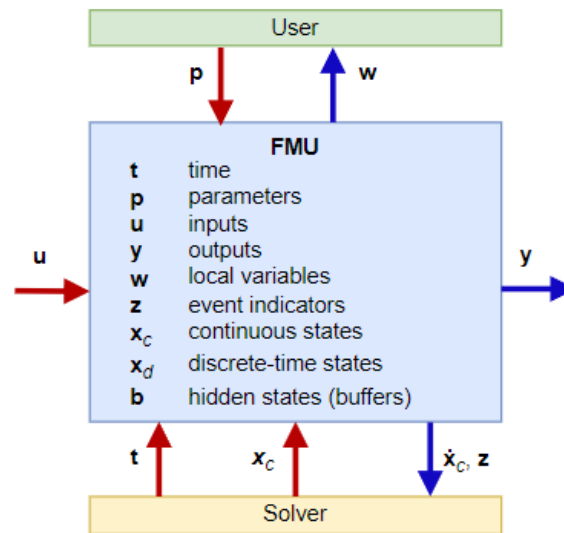


Figure 2-7. Model Exchange diagram [17].

ME is the most basic version of an FMU and primarily used for importing/exporting between independent simulation platforms. The main benefit is the control over the solver's behavior in addition to not requiring a license from the exporting platform to run. Consequently, the drawback is the requirement for developing a solver capable of running the FMU.

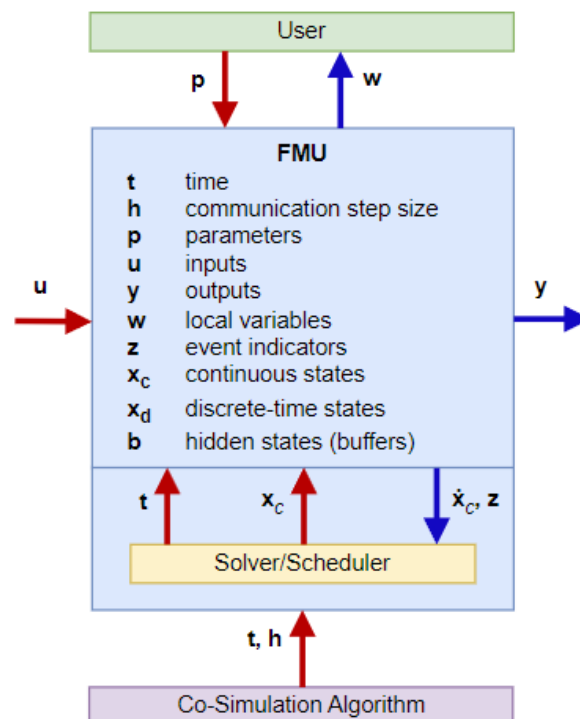


Figure 2-8. Co-Simulation diagram [17].

CS directly provides the opposite approach with the exporting platform's solver integrated into the FMU. While requiring a license on hand, this allows the user to implement the FMU outside of a standard simulation platform.

Naturally, a CS model type was chosen due to the significantly reduced requirements for initial testing. However, this may need to be re-evaluated in the future dependent on IGNITE's final integration level within Amesim.

2.3.3. FMI Integration in Amesim

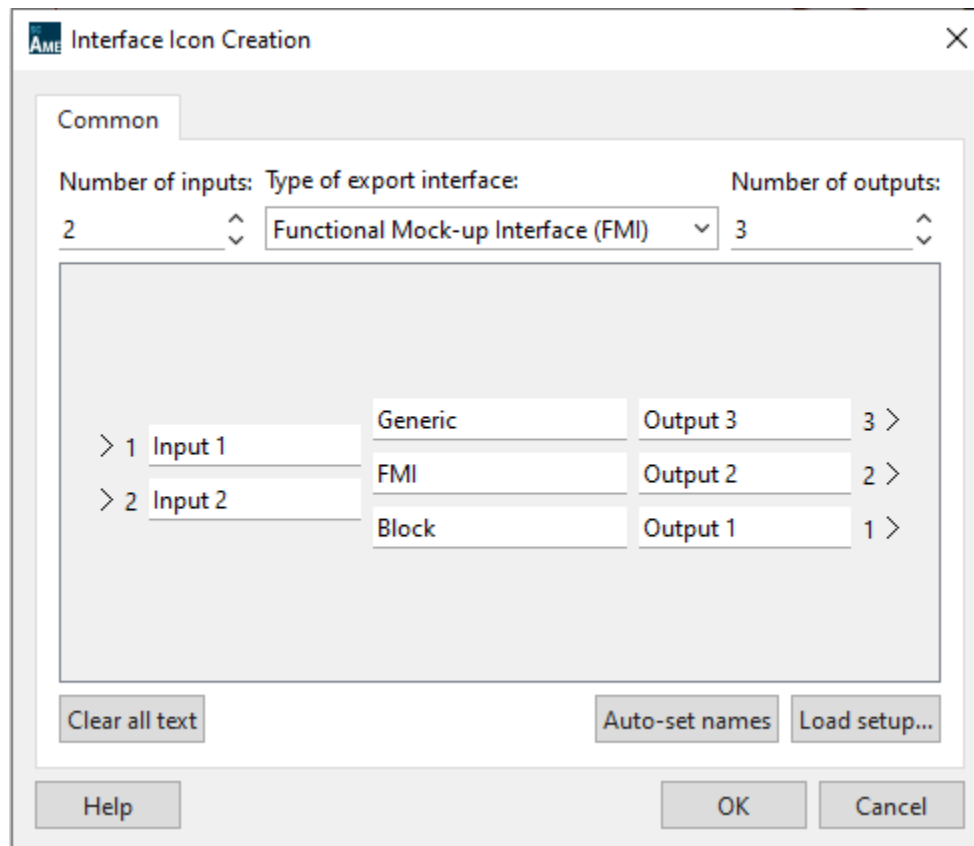


Figure 2-9. Amesim FMI interface component.

Amesim's FMI component block defines the inputs and outputs for the FMU. When implementing the FMU, the outputs of the component become the inputs into the FMU and vice versa.

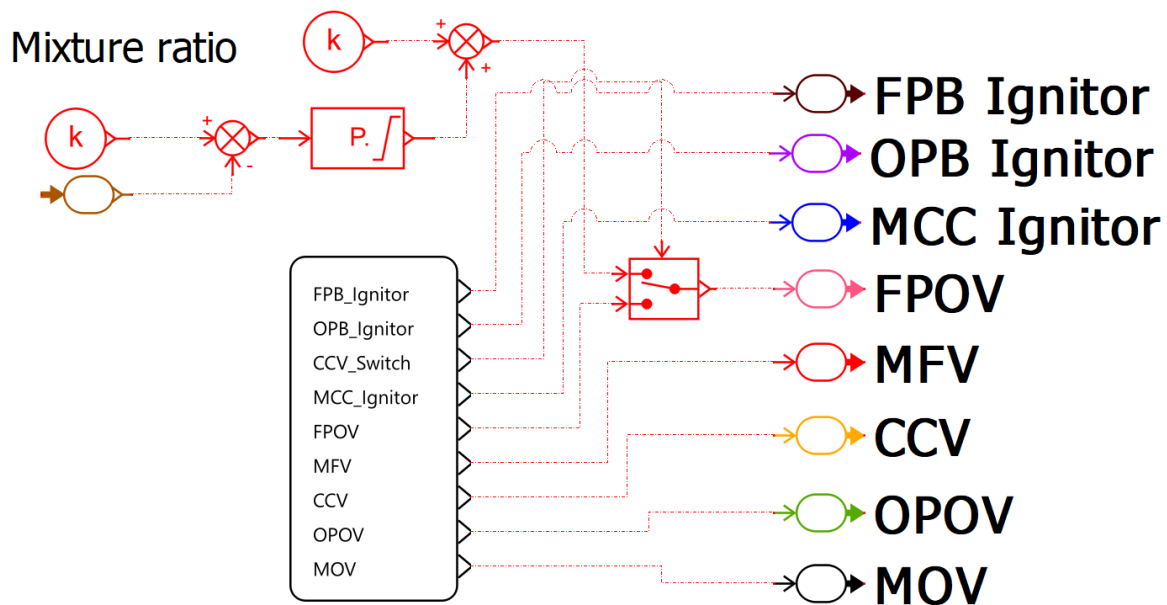


Figure 2-10. RS-25's engine controller.

Typically, the model returns some output to be used in a larger simulation model. However, the agent needs access to all of the state variables while maintaining the system of ODEs. Therefore, as seen in Figure 2-10, there are no inputs into the FMU component block. Instead, the visibility of the FMU is set to high which allows the agent access to all state variables in the model without explicitly labeling them as outputs into the agent.

A quick aside, the MCC, OPB, and FPB Ignitors refer to the switches on each CC which ignites the products inside the CC and starts combustion. In the demo model, these are set to known values, however the agent will need to determine the stability of each CC before enabling the ignitor. This allows for additional flexibility in the agent's solution and potentially improving the performance compared to the reference ignition sequence.

3. REINFORCEMENT LEARNING AGENT

Reinforcement Learning (RL) is a semi-supervised type of machine learning where an agent interacts with some environment through actions and receives a state and rewards from that environment [1].

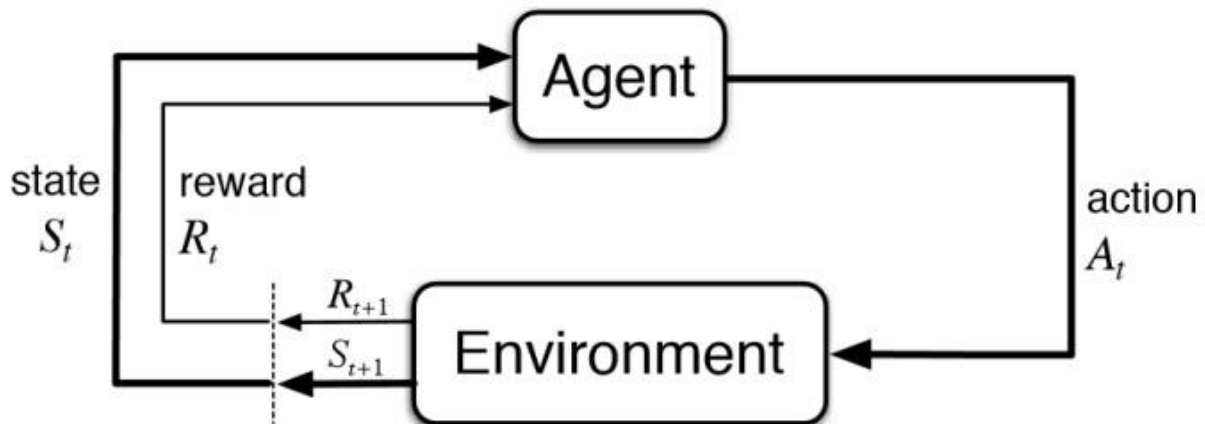


Figure 3-1. generic RL diagram [18].

RL has the benefit of generating its own training data in addition to approximately matching how humans learn through trial and error. Given the inherent lack of training data as well as trying to maintain a generalizable approach, RL is suited as a solution for this project.

3.1. Soft Actor-Critic Architecture

Out of the multitude of deep RL architectures, the Soft Actor-Critic (SAC) architecture was chosen due to its stability in large state spaces in addition to the maximal training efficiency [4].

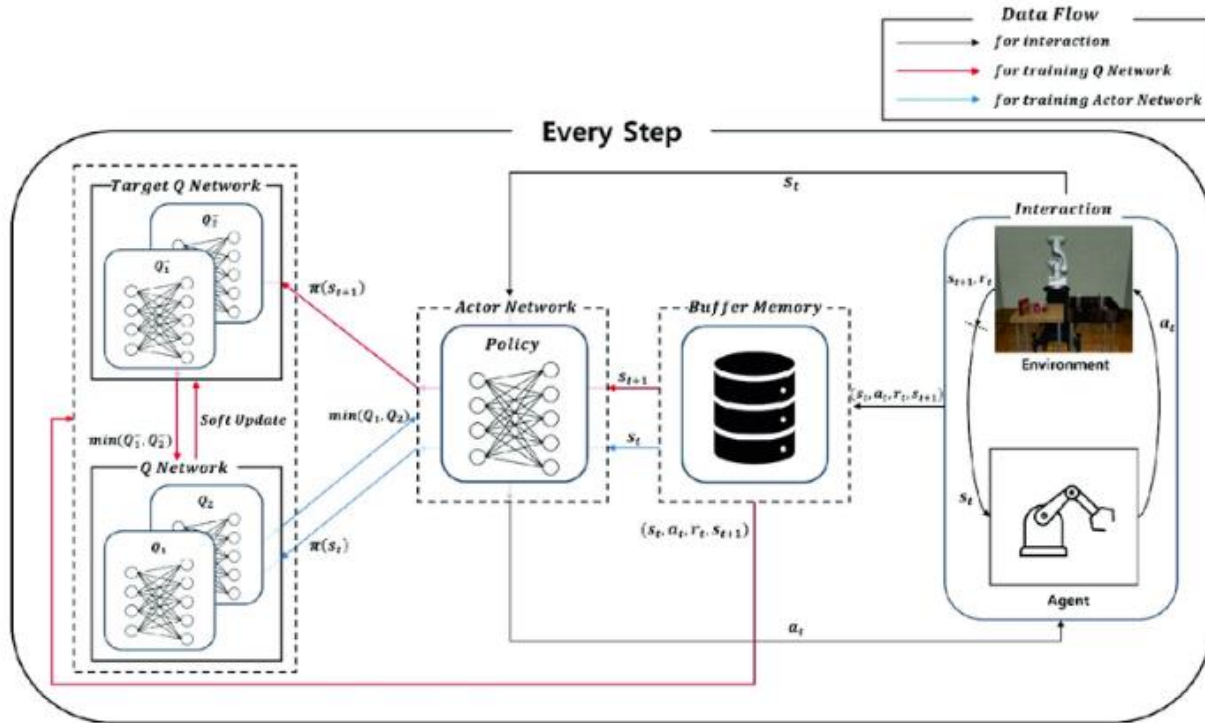


Figure 3-2. Soft Actor-Critic overview diagram [19].

This is achieved through the augmentation of an Actor-Critic architecture where an entropy parameter is introduced into the training procedure. The entropy of the system is related to the balance between exploration and exploitation of the actor's current policy and it allows for the dynamic exploration of the state space until an optimal policy is found. Importantly, the implementation does not detract from the critic's understanding of the environment which allows it to utilize all learned experiences, including those gathered with sub-optimal policies.

3.1.1. Actor Network

The actor is a standard Deep Neural Network (DNN) which makes stochastic actions compared to standard discrete actions. To retrieve an action, the associated LRE state is sent forward through the NN layers and the actor returns two parameters: mu and sigma.

Given the stochasticity of the agent, mu and sigma represent the parameters of a standard normal distribution for each valve. Mu represents the mean of the distribution while sigma represents the standard deviation. Given the standard deviation must be positive, non-zero, and not large ($>>1$), the sigma parameter is clamped to a range of some epsilon ($1e-6$) to 1.

The resulting distributions are then sampled via the [reparameterization trick](#) to enable differentiability for backpropagation (training) later. The samples for each distribution are the

actions the agent will take; however, they range from $-\infty$ to ∞ which cannot be easily translated to a physical system. Therefore, a hyperbolic tangent function is applied to map the action space from -1 to 1 which represents the maximum closing and opening rates of the valves respectively.

Additionally, as a feature of SAC, the joint log probabilities between the distributions of all the valves are computed (with a correction for the hyperbolic tangent squashing) which represent the total entropy of the actions. The agent then utilizes the entropy to inform itself on the viability of a current policy against the possibility of a more optimal policy.

3.1.2. Critic and Target Critic Network

The critic is a standard Deep Quality Network (DQN) which evaluates the actor's actions against a state to determine a "quality value" of that state-action pair. The quality value (q-value) acts as a reward for the actor to maximize through learning an optimal policy.

Given the critic acts as the judge for the actor, it needs to be training on the state space to give an accurate analysis of a state-action pair. In other words, while the actor learns the behavior of its actions against a state, the critic learns the evolution of the states given an action. Therefore, the critic needs a value function to learn the ground truth behavior of the environment.

SAC utilizes a separate target critic network in order to act as the value function which is updated via a rolling average of the previous critic weights. This stabilizes training significantly by preventing the critic from losing valuable information, even from suboptimal policies or inaccurate q-values. This has a few drawbacks, especially when the critic over-trains over a specific reaction in the system, however it proves to be overall more advantageous to the agent's training.

3.1.3. Entropy Parameter

The final learnable parameter is the entropy term (sometime referred to as temperature term) which dictates the ratio between exploration and exploitation of a policy. This term is applied to the entropy (joint log probabilities) of the actor's action to modify their impact in the training procedure. As the entropy term approaches zero, the entropy is no longer a factor and the agent essentially follows the Actor-Critic architecture or fully exploitive. As the entropy term increases, the agent attempts to explore more of the state space.

3.2. Training Structure

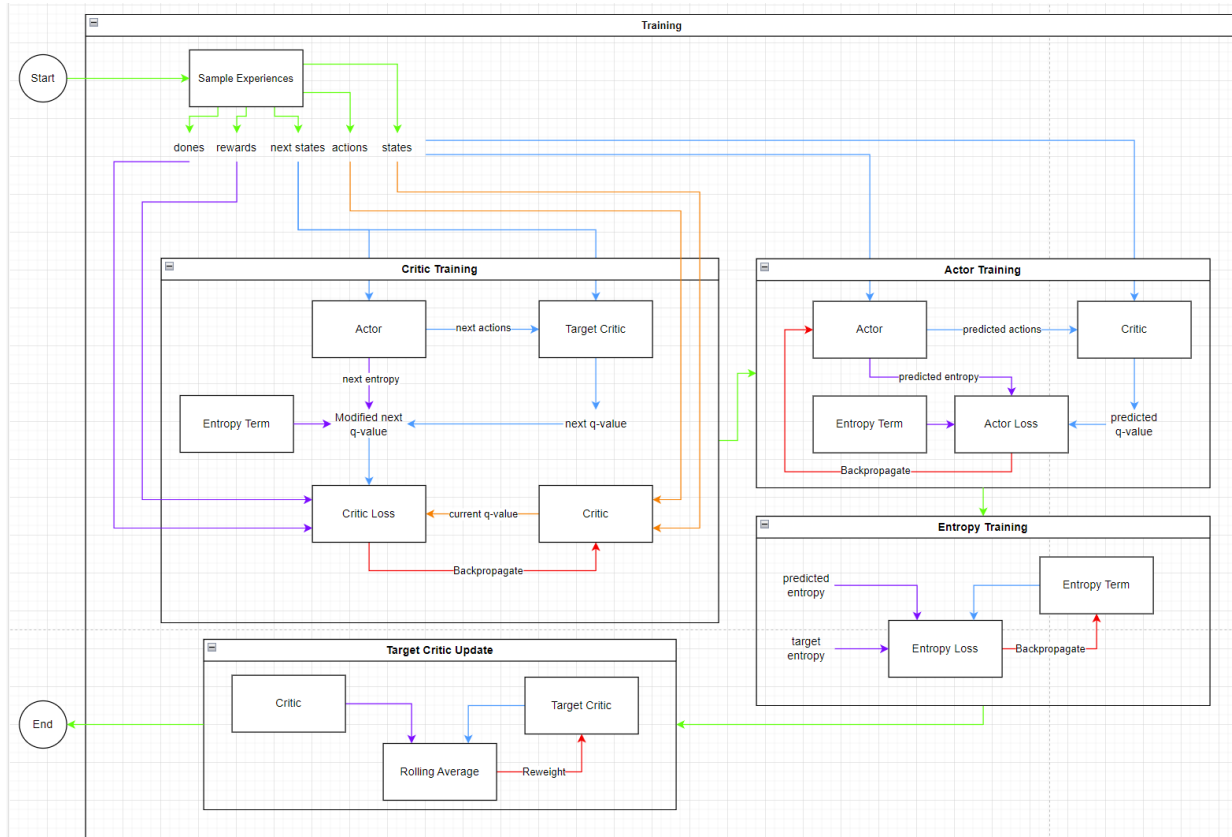


Figure 3-3. Soft Actor-Critic overall training diagram.

The following sections will explain the overall training procedure for the SAC architecture by breaking Figure 3-3 into individual sections in chronological order. This is important as updates are ordered to promote a policy-free based approach which requires the information to flow from the value network to the critic networks to the actor networks to any remaining parameters.

An important note about the colors in this section's diagrams:

- Green: Chronological flow in overall diagram
- Blue: Primary flow in each subsection
- Orange: Secondary flow in each subsection
- Purple: Modifies attached component
- Red: Updates attached component (always occurs last)

Before any networks are trained, the experiences from the agent's internal memory must be parsed into the following separate components:

- States

- Actions
- Next States
- Rewards
- Dones

The original of these components are specified in section 4.3. States contains the initial state of the environment for a given experience. Actions contains the agent's actions given that initial state. Next States contains the resultant environment state after the actions are applied. Rewards contain the reward associated for the agent actions and resultant environment state. Dones contains the internal flag whether the simulated run ended.

An aside, each of the components above are arrays pulled from many individual experiences from the simulation loop. The calculations in the following sections are using the components in array form and then combining them together for the backpropagation or update step.

3.2.1. Critic Training

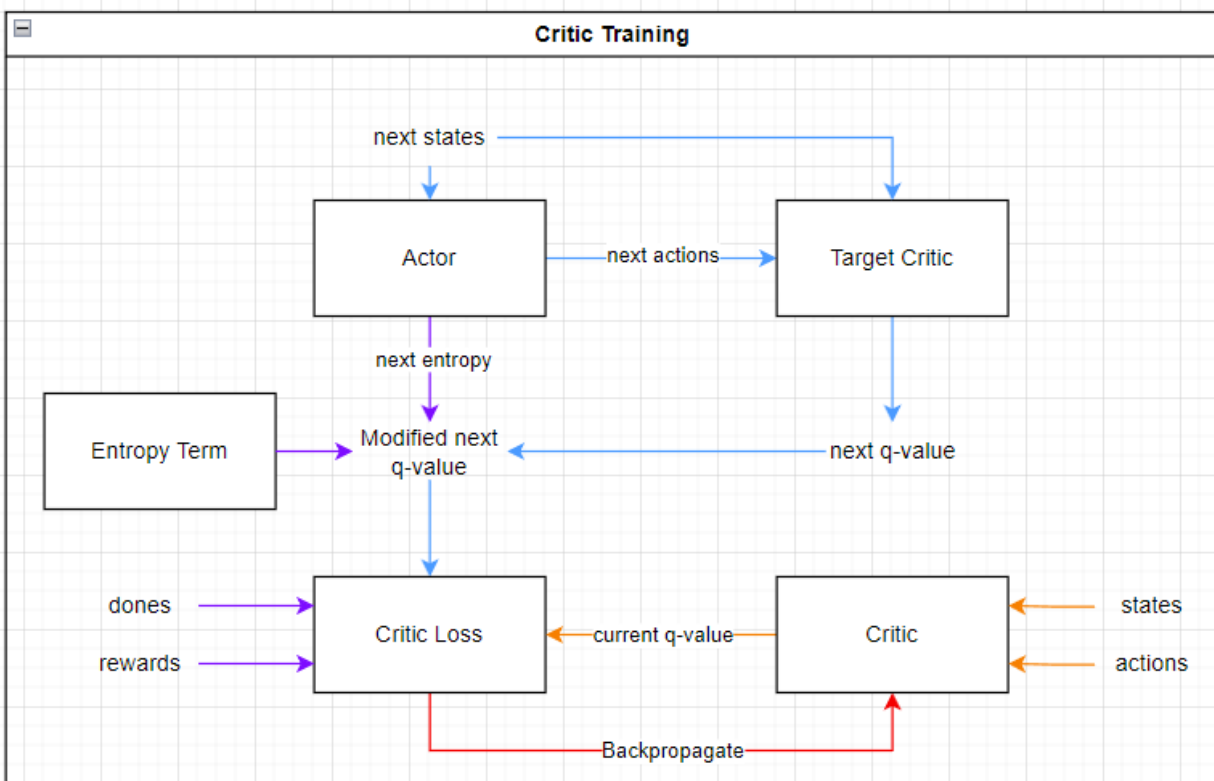


Figure 3-4. critic training diagram.

The first event in training is the calculation of the value function and applying it to the critic network. The value function is computed via [Bellman's Equation](#) which requires determining the future value to update the current value.

This starts with the actor network passing in the next states to determine the next actions it would have taken in the simulation loop. The next state-action pair is then passed into the target critic network to determine the next q-value.

Currently, the next q-value is directly linked to environment which would be passed into Bellman's Equation in an Actor-Critic architecture. However, SAC decreases the next q-value by introducing entropy, under-valuing the evaluated quality of the next state-action pair. By under-valuing the quality, the agent is essentially leveling the quality of the current policies to prevent immediate exploitation of the current optimal policy.

Additionally, the next q-value is modified separately from the target critic, meaning the weights that resulted in the next q-value remain untouched. This allows the agent to decrease the entropy parameter and recover the exploitation of the optimal policy without having to significantly retrain.

$$V(s_i) = R(s_{i+1}) + \gamma V(s_{i+1})$$

Equation 3-1. simplified Bellman's Equation.

The value function is then calculated following Equation 3-1 where s_i represents the initial state, s_{i+1} represents the next state, and $V(x)$ represents the value function given some state x . The future value function, $V(s_{i+1})$, is the modified next q-value computed by the target critic network. The ground truth of the behavior is introduced via the reward associated with the next state and is represented by $R(s_{i+1})$. The discount factor, γ , reduces the value associated with the modified next q-value to promote maximizing current rewards over future rewards.

Separately, the initial state-action pair is passed through the critic network to determine the current q-value. The squared difference (MSE) between the computed value function (target q-value) and the current q-value is the critic loss. Finally, the critic loss is backpropagated through the critic network to update its parameters.

The critic loss represents the difference between true behavior of the environment and what the critic network believes the behavior of the environment to be. Therefore, the goal of the agent is to minimize the critic loss as that would signify the critic network is able to accurately predict the resulting quality of a given state-action pair.

3.2.2. Actor Training

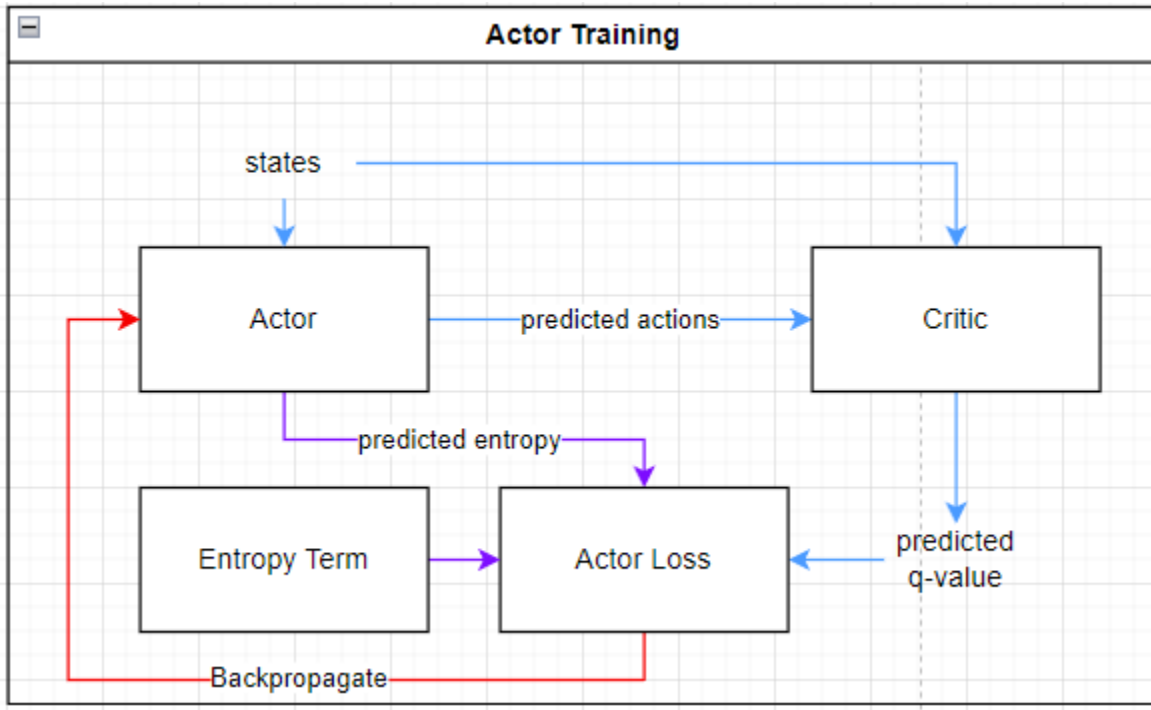


Figure 3-5. actor training diagram.

Using the newly updated critic network, the actor's loss can be calculated. The initial state of the environment is passed through the actor to generate its predicted actions and associated entropy. The critic evaluates the initial state-predicted action pair and produces a predicted q-value.

Similar to the critic network training, the q-value is modified to include the entropy of the actions to under-value the quality of the current policy. Given the modified q-value is directly used as the actor's loss, new properties arise from introducing the entropy.

The goal of the actor remains the same which is to minimize its loss function which in turn represents maximizing its quality. However, the entropy modification decreases the q-value which increases the loss function. The result is that the loss function is now bound from $-\infty$ to ∞ instead of strictly negative. Therefore, a positive actor loss represents promoting exploration while a negative actor loss represents exploiting the current policy.

This representation is confirmed during backpropagation where positive losses correlate to negative gradients, essentially rolling back parameters to find new paths to explore. Negative losses correlate to positive gradients, reinforcing the current policy by strengthening the parameters.

3.2.3. Entropy Training

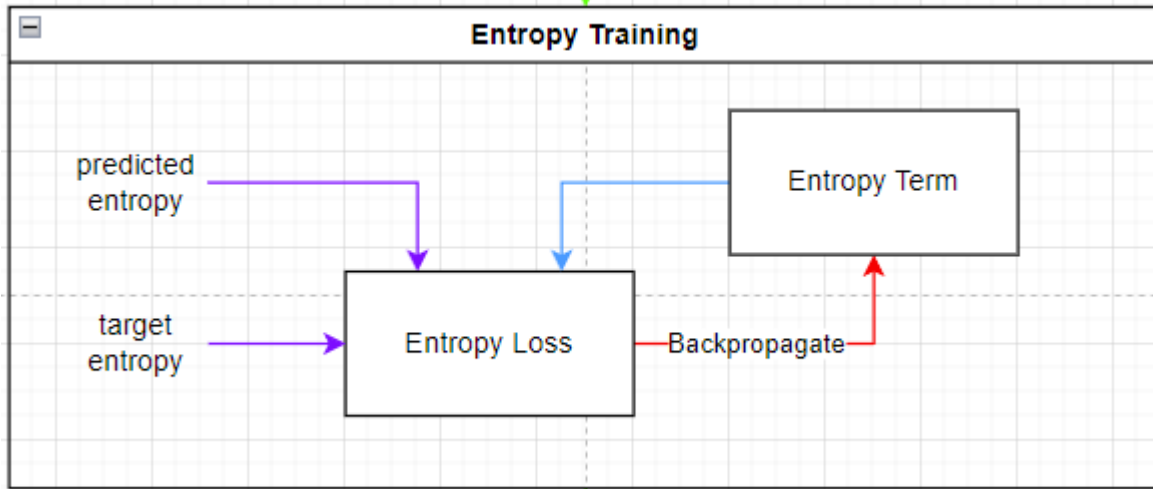


Figure 3-6. entropy training diagram.

For this project, the entropy term is a trainable parameter to promote dynamic exploration of the policy space. A target entropy term is utilized to promote the training to converge towards an optimal policy, typically the negative of the action space. The entropy term must remain positive so the difference between the target entropy and predicted entropy from the actor training is multiplied by the log of the entropy term. The resulting value is the entropy loss which is backpropagated through the single node.

When turned into a trainable parameter, the entropy term has a host of unique behaviors which depend significantly on factors like rewards, actor, and critic behavior. Further details with examples are discussed in section 5.2.

3.2.4. Target Critic Update

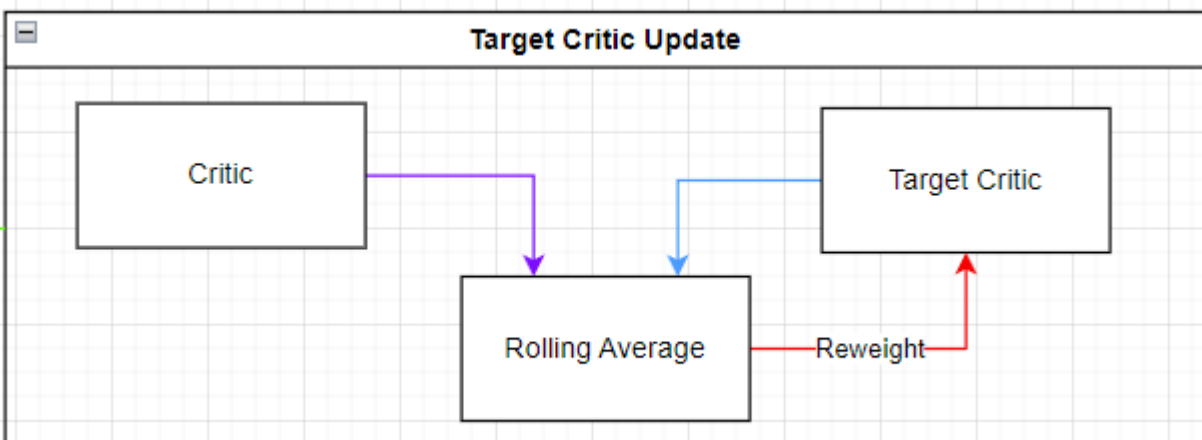


Figure 3-7. target critic update diagram.

Finally, the target critic network is updated with the new critic network parameters modified by a parameter τ ($5e-2$). The critic parameters are multiplied by τ while the target critic parameters are multiplied by $1 - \tau$ and both are summed together to get the new target critic network parameters.

An aside, the target critic and critic networks are actually 4 networks in total, 2 for each type. This is done to prevent an overestimation of the q-value which is typical for RL agents to experience. The minimum of the two networks is used at each step to reduce the overestimation as much as possible. Additionally, this stabilizes the q-value between training events as oscillatory behavior is common when the agent approaches zero loss but the learning rate remains large.

3.3. Reward Structure

The reward encoder consists of a set of prime checks, a reward function, and a stability check. The current state variables, valve positions, current time, stop time, and a set of Booleans indicating the ignition prime status of each combustion chamber (CC) are used to compute the rewards. A set of scaling factors is defined by a user defined value and used to weight the importance of each reward on stability.

The reward encoder iterates over each CC and increments either transient or ignition rewards based on the prime check. Individual rewards are assigned as a scaled percentage of the target, with negative rewards assigned for violations [8]. The total rewards are equal to the sum of all the individual rewards.

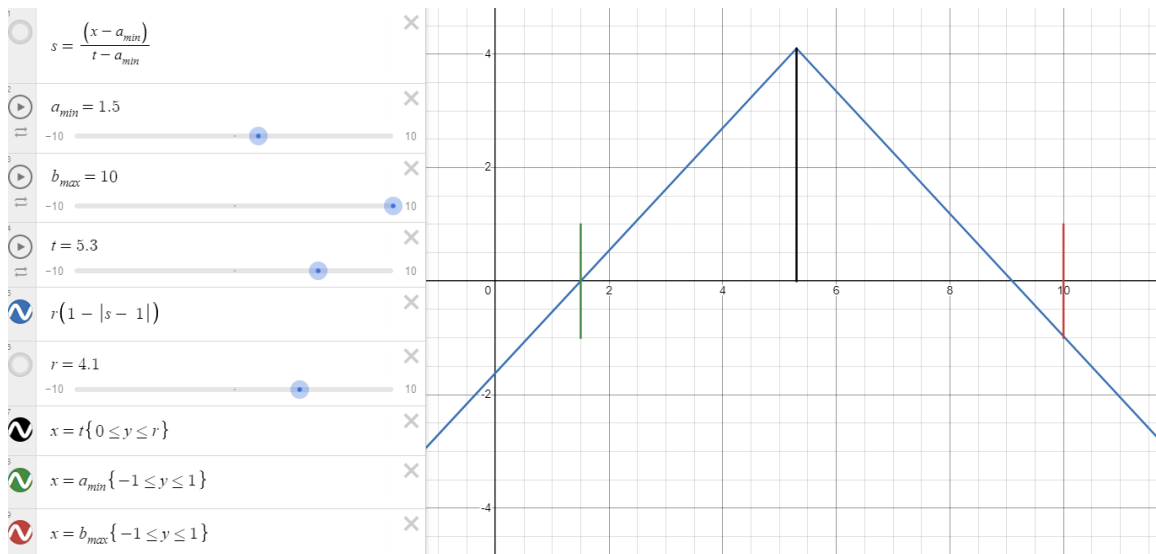


Figure 3-8. target value reward function.

3.3.1. Prime Checks

Each CC has its own ignition check. The function first checks if the chamber has already been primed. If not, then the function checks that the MR is less than 1.5 times the target value [4]. The result is used to determine whether the reward encoder should assign transient or ignition rewards for each chamber. The correct MR ensures an oxygen rich environment, which is necessary to avoid explosions in the CC and other failure modes such as engine unstart [6].

3.3.2. Transient Rewards

If the LRE is in the transient state, individual rewards are assigned for the CC MRs and pressure increases. The current MR of the CC is compared against a target and rewards are assigned based on closeness to the target. The MR reward encourages the agent to approach the prime check conditions so the simulation can progress. The current pressure of the CC is compared to previous pressure values. The difference between the current and previous pressures are scaled and rewards are assigned accordingly. In the transient stage, pressure should be continuously increasing, rather than staying around target [2]. If the MCC is not primed, individual rewards are also assigned for the Mach number in the nozzle throat approaching a choked value. This is necessary for stability in the ignition stage [12].

3.3.3. Ignition Rewards

If the LRE is in the ignition state, individual rewards are assigned for the CC pressure and temperature according to how close they are to the target. Ideal temperatures and pressures affect thrust performance but are simpler for the agent to control simply by opening and closing valves [12]. If the main combustion chamber is primed, rewards for delay pressure and temperature are calculated. The pressure and temperature deviation are equal to the reward multiplied by a delay power. The reward is assigned as a product of the deviation and percent of the total time the episode has progressed. Delay pressure rewards are also assigned for the pre-burners. Delay rewards discourage the agent from overshooting the goal and exploiting temporary peaks in rewards [14]. If all of the CCs are primed, individual rewards are assigned for valve positions and pump rpms according to closeness to some target. The valve position reward discourages the agent from opening certain valves to the maximum position and closing certain valves to “cheat” the rewards. Instead, the valves are rewarded for increasing and staying around the closed-loop positions, while allowing for some movement around the target which is necessary for stability. The pump rpm reward is directly related to the pressure rise and head rise across each pump, which relates to pump efficiency [9]. The reward also acts as a reasonable check for leakage, which would cause the pressure to fall much shorter from the target [7].



Figure 3-9. target delay reward function.

3.3.4. Violations

Violations are assigned as negative rewards (penalties) and calculated by the state variables handler. The variable handler compares each system variable to its hard limit and assigns a penalty when a hard limit is exceeded [8]. These hard limits either correspond to physical limits, such as 0 K, or operational limits for each component, defined by the user. The total penalty is the sum of all individual penalties and applied as a negative reward in the reward encoder.

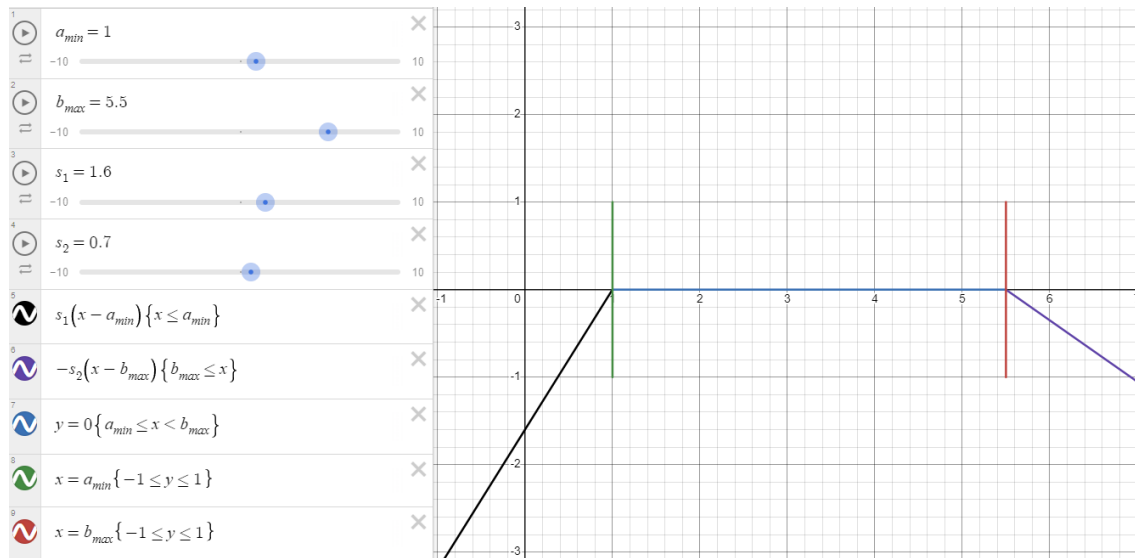


Figure 3-10. violation penalty function.

3.4. Agent Hyper-Parameters

Agent hyperparameters refer to any manually tunable term in the agent that affects its performance and behavior during training. Optimization of them can reduce maximal policy search times, prevent agent divergence, and increase training efficiency.

3.4.1. Learning Rate

A feature of all DNNs, the learning rate (LR) modifies the gradient in the backpropagation to damp the parameter weight update. The primary intent is to prevent overshooting global maximums while encouraging exploration of the state space. LRs tend to vary between $1e-5$ and $1e-1$ with the authors of the original SAC paper choosing $3e-4$ as an initial heuristic.

Simpler environments with well defined global maximums can opt for an initially large LR to speed up training, however more complex environments require lower LRs to prevent losing valuable information through large gradients. Some amount of tuning will always be required to find the optimal LR, but the benefits typically overshadow the effort required.

3.4.2. Entropy Scale

The target entropy can be scaled up or down through the entropy scale factor. A higher entropy scale promotes a more exploitative policy compared and vice versa. The entropy scale can therefore be assumed as a measure of randomness for a converged policy. In section 6.1.1, an entropy scale of 4 was utilized to promote a final deterministic policy.

3.4.3. Training Threshold

The training threshold is an integer to determine how many experiences are required to be gathered before training the agent. The simpler the environment, the less experiences are required to determine the overall behavior and therefore the lower the training threshold. The opposite is also true with more experiences being required for complex environments and therefore a higher training threshold. This project currently uses 24,000 experiences and samples 2,400 experiences; however, it is likely to change due to improvements listed in section 7.

3.4.4. Network Layer Size and Depth

A DNN's layer size and depth refers to the number and arrangement of the nodes in the DNN's hidden layers. Size is the number of nodes in a given layer and depth is the total number of layers in the system. Depending on the complexity or form of the environment, different sizes and depths are used with smaller ones for simpler environments and large ones for complex environments. This project specifically uses 256 nodes and 2 layers as an initial heuristic, but more testing will likely result in a need to increase both size and depth.

4. IGNITE STRUCTURE

IGNITE has three main capabilities: an interface with the FMU, a training loop that returns the valve control sequence guess, and a set of post processing capabilities. IGNITE interfaces with the FMU through variable handling to standardize units and magnitudes across all environment variables. The training loop is comprised of two loops. The step-by-step processing occurs in the individual episode loop, while the general agent training processing occurs in the main loop. As episodes are simulated and stored, the agent will eventually fill up its internal memory and starts training on the experiences. The agent repeatedly trains in episodes until it reaches the desired initial ignition sequence. Post processing is done to display the losses, rewards, and valve control sequences so the user can come to their own conclusions about the results and adjust the training parameters to improve the model.

4.1. Variable Handling

To properly interact with its environment, the RL agent needs a way of interfacing with the FMU. The [FMPy library](#) provides the API calls into the FMU, yet the API functions are obtuse and do not store the necessary information to calculate rewards. Therefore, a StateVariable class is implemented to solve both issues.

```
class StateVariable:
    """
    Class used to hold variable properties to interface between the FMU and the agent.

    Usage Structure:
        {object_name}.{member} = 1
        foo = {object_name}.{member}

    Variable Structure:
        *Name = User-defined name of state variable
        *Component = Primary component of LRE state variable represents
        *Agent State = Defines usage in agent's state space or for rewards
        *ReferenceName = FMU-defined name of state variable
        *ReferenceValue = Value used to find the associated FMU variable
        ReferenceUnits = FMU-defined units
        *ScaleMinValue = Expected minimum value of the variable
        *ScaleMaxValue = Expected maximum value of the variable
        HardMinValue = Physical minimum value of the variable
        HardMaxValue = Physical maximum value of the variable
        TargetValue = Target value of the variable
        *UserUnits = User-defined units
        CurrentValue = Value of the associated FMU variable
        CurrentScaledValue = CurrentValue rescaled via lerp given MinValue and MaxValue
        CurrentDerivative = Average forward derivative of current value in a specified window size
```

Figure 4-1. StateVariable class structure and members.

Each desired variable is initialized as a `StateVariable` object with its behavior defined via several parameters.

The `Name`, `Component`, and `Agent State` define the object's usage in the system. `Name` is a user-definable ID for referencing in other portions of the codebase. `Component` specifies which LRE component the object is primarily associated with and is used for determining state violations. `Agent State` determines if the object is a learnable state passed into the agent. Given the 131 unknown variables and the high redundancy (MFR on both sides of the same pipe), limiting the amount of states passed into the agent will decrease training time and improve stability.

The reference members are used for API calls. `ReferenceName` and `ReferenceValue` are both valid IDs for a variable in the FMU. Only one needs to be defined in order to call that variable. `ReferenceUnits` define what units the FMU utilizes for that variable. In order to prevent unnecessary conversion within the rest of the code, all `ReferenceUnits` are converted to SI units through the `UserUnits` member.

The variables typically have significantly varying magnitudes, such as temperature compared to pressure. If passed directly into the agent, the differing magnitudes would cause significant numerical instability which leads to erratic behavior [5]. Therefore, `ScaleMinValue` and `ScaleMaxValue` are used to linearly map the expected range of a variable to between 0 and 1. When the min/max values represent physical limits, the `HardMinValue` and `HardMaxValue` are used instead. These function the same as their scale counterparts, but penalties are applied if they are exceeded. Having two different scaling types helps account for the unique behaviors of a variety of variables without hard coding it, promoting generalizability.

If a variable is a design parameter, the `TargetValue` is set to the known target value. This allows the agent to learn an optimal policy faster while also reducing training time and increasing training stability.

The `CurrentValue`, `CurrentScaledValue`, and `CurrentDerivative` contain the variable's value which can be called in to other files and updated. The derivative is calculated through a forward finite method with an averaging window of 5 as the default. The window size is an input parameter when retrieving the derivative, so long- or short-term behavior can be analyzed.

4.2. Main Loop

The main loop starts by initializing the FMU and simulating an episode. The process describes in the episode loop is completed and the reward encoder checks if the system is stable (refer to section 7.1). If so, a suitable ignition sequence has been found and the program is complete. Otherwise, the agent memory length is compared to some training threshold and the agent is trained when the memory is full. In either case, the FMU is terminated and the results and models are saved. The FMU is again initialized and the process repeats until a stable ignition sequence has been found.

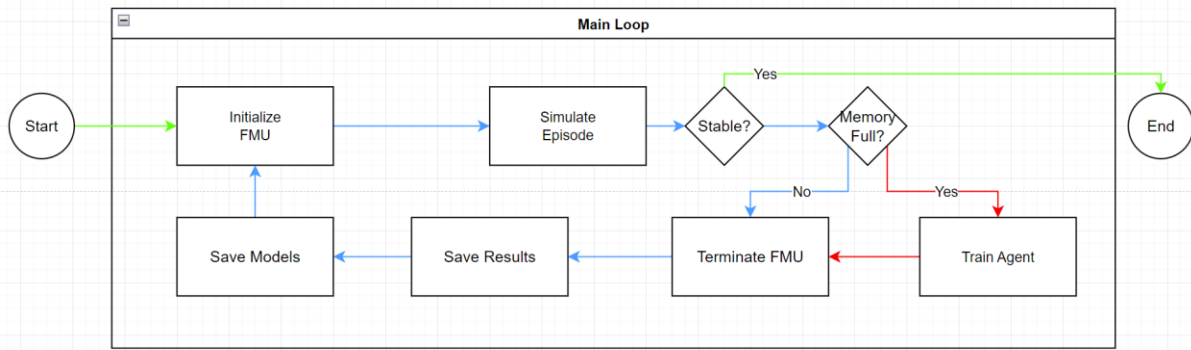


Figure 4-2. main loop diagram.

4.3. Episode Loop

A single episode is defined to from the ignition sequence start to some end state where the engine reaches some steady state condition. Inside of each individual episode, the agent is in a state where no learning occurs. In each iteration, the program grabs the most recent state from the FMU model and the variables defining the initial state are scaled to the encoded state, which removes variable units and magnitudes so they can more easily be interpreted by the agent. The encoded state is passed forward through the actor and the agent returns a set of actions to change the valve positions. The new positions are calculated and sent into the FMU which is incremented one time step. The FMU returns the new states and the rewards are calculated based on improvements by the actions from the initial state. All information gathered in this single step is stored as an “experience” in the agent’s memory. The process is repeated until either the system is stable or the stop time has been reached. This is a single episode.

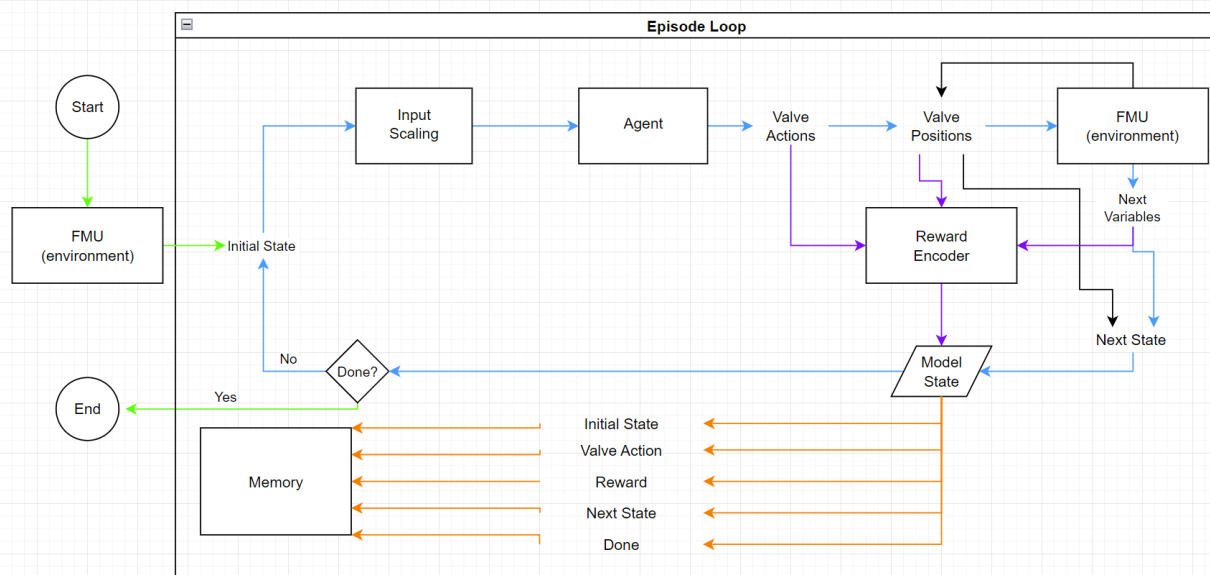


Figure 4-3. episode loop diagram.

4.4. GUI

A graphical user interface (GUI) was created using the PySide6 package. The GUI consists of three main features: an input window to control the main process inputs, a log window to display runtime messages, and a plot window to display post processing scenes.

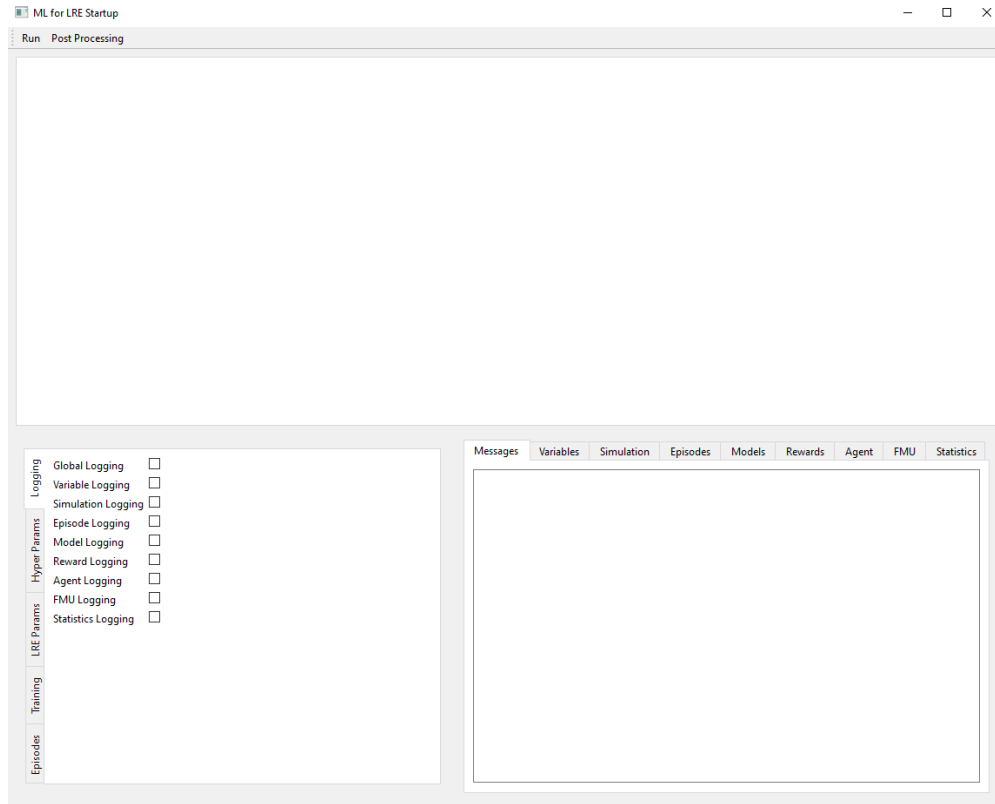


Figure 4-4. GUI main window.

The GUI is able to run the main loop as a threaded process when the “Run” button is pressed using the inputs in the input window.

4.4.1. Input Window

The inputs for the main process can be defined in the lower-left section of the main window. The specified defaults are auto populated in their respective fields. The main loop is run with the values in the input fields when the “Run” button is pressed and the parameters cannot be changed without restarting the main loop.

Tab	Parameter	Value
Logging	Learning Rate	1e-4
	Reward Scale	1
Hyper Params	Entropy Parameters	0.5
	Entropy Scale	4
	Temporal Delay Offset	0.05
LRE Params	LR Patience	20
	LR Cooldown	20
	Agent Warmup	1000
Training	LR Mode	min
	LR Scale Factor	0.5
Episodes	Discount Factor	0.99

Figure 4-5. input section of main window in the GUI.

4.4.2. Log Window

While the main loop is running, any messages are logged in the log section of the main window. Each tab corresponds to a logging check input by the user. The current date and time are appended to the start of the message in MM/DD/YYYY, HH:MM:SS format.

Tab	Message
Messages	04/30/2024, 13:49:19 - Reward: 0.0
	04/30/2024, 13:49:19 - Variable Name: FPB_Mixture_Ratio
	04/30/2024, 13:49:19 - Current Value: 0.0
	04/30/2024, 13:49:19 - Reward: 0.0
	04/30/2024, 13:49:20 - Variable Name: MCC_Mixture_Ratio
	04/30/2024, 13:49:20 - Current Value: 0.5498504298531984
	04/30/2024, 13:49:20 - Reward: 0.5498504298531984
	04/30/2024, 13:49:20 - Variable Name: OPB_Mixture_Ratio
	04/30/2024, 13:49:20 - Current Value: 0.0
	04/30/2024, 13:49:20 - Reward: 0.0
	04/30/2024, 13:49:20 - Variable Name: FPB_Mixture_Ratio
	04/30/2024, 13:49:20 - Current Value: 0.0
	04/30/2024, 13:49:20 - Reward: 0.0
	04/30/2024, 13:49:20 - Variable Name: MCC_Mixture_Ratio
	04/30/2024, 13:49:20 - Current Value: 0.6555633322488789
04/30/2024, 13:49:20 - Reward: 0.6555633322488789	
04/30/2024, 13:49:20 - Variable Name: OPB_Mixture_Ratio	
04/30/2024, 13:49:20 - Current Value: 0.0	
04/30/2024, 13:49:20 - Reward: 0.0	
04/30/2024, 13:49:20 - Variable Name: FPB_Mixture_Ratio	
04/30/2024, 13:49:20 - Current Value: 0.0	
04/30/2024, 13:49:20 - Reward: 0.0	

Figure 4-6. log section of main window in the GUI.

4.4.3. Plot Window

The post processing button in the upper toolbar opens another window prompting the user to specify an episode number.

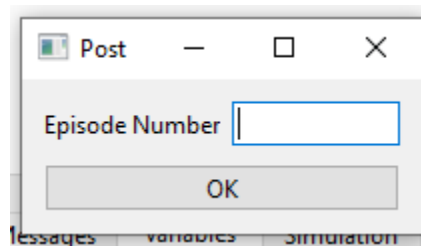


Figure 4-7. post-processing window.

The rewards and ignition sequence from the specified episode number are plotted in the upper section of the main window. Each plot is opened in a different closable tab.

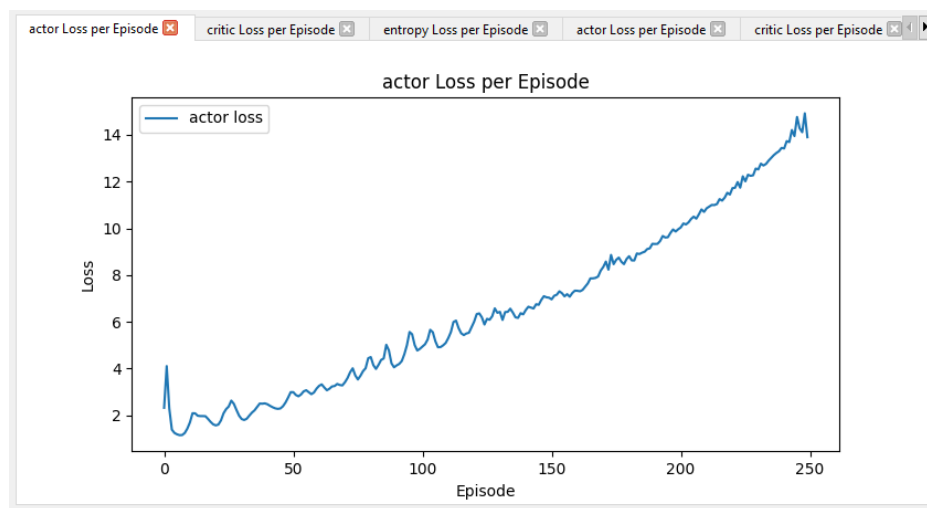


Figure 4-8. plot section of the main window.

4.5. Post-Processing

Post processing is controlled from the GUI using the “Post Processing” button. A data handler is used to convert data between csv files and data frames. In the main loop, the data handler saves loss values, rewards, and ignition sequences in csv files. In post processing, the data handler is used to read the csv files, convert the data back to data frames, and plot the data. The post processing method in the GUI calls plot functions from the data handler, which reads the csv files, passes the data back to plot window, and is displayed in a new tab.

5. VERIFICATION AND VALIDATION

Ensuring major subsystems function as intended is vital writing safe and functional code. As major subsystems are created, appropriate tests and analysis shall be documented below.

5.1. Environment

The environment's internal state is not directly accessible through standard python functionality, therefore extensive testing was required to ensure all functions worked as intended. The goal of this testing was to replicate a reference ignition sequence static fire with the FMU and compare the results to the ones produced by Amesim.

5.1.1. FMU Variables

Using the FMU's model description, the list of all internal variables was constructed to determine how the FMU was organized by Amesim. Variables are categorized into three primary groups: Input, Output, and Local.

```
Name: dynamic_rotary_node_3.p1__omegat, Ref Val: 781, Unit: rev/min
Name: lp_turbine.torque, Ref Val: 782, Unit: Nm
Name: tpf_adiab_chamber_7.p2, Ref Val: 783, Unit: barA
Name: tpf_adiab_chamber_7.rho2, Ref Val: 784, Unit: kg/m**3
Name: tpf_adiab_chamber_7.cnum2, Ref Val: 785, Unit: tpfnull
Name: lp_turbine.dmh3, Ref Val: 786, Unit: W
Name: lp_turbine.dm3, Ref Val: 787, Unit: kg/s
Name: lp_turbine.hin, Ref Val: 788, Unit: kJ/kg
Name: lp_turbine.pratio, Ref Val: 789, Unit: None
Name: lp_turbine.prc, Ref Val: 790, Unit: None
```

Figure 5-1. example of FMU variables.

Input and Output variables associated to the FMI block ports created in Amesim. Under lower internal state visibility settings during FMU export, these two categories would be the only ones visible under the model description. The Local variables refer to the component variables in the model and are only available under the highest visibility settings.

The variables in the model description do not exactly match the variables displayed in Amesim with some variables only available in Amesim (MCC fuel port's fluid density) and others only available in the FMU (MCC fuel port's MFR). This has not affected reward structure modeling; however, it may cause issues in the future.

5.1.2. General FMU Simulation

Initializing the FMU requires the creation of the FMU3Slave object which needs to be instantiated with simulation parameters. The most noteworthy parameter is loggingOn which prints the FMU's action out to the console. This is disabled by default and important information about the state of the simulation will not be visible unless enabled.

Time advances through a loop which repeats until the designated stop time is reached. The FMU's internal steps do not match the external steps (communication points) of the loop primarily due to internal solver actions. Therefore, the FMU will always take the same time to complete a run regardless of the communication point step size. Furthermore, the amount of time per each step can vary significantly as the FMU uses different solver types dependent on the internal environment state. These range from ODE integrators ([RK](#) or [AB](#)) to 1D CFD solvers. The corresponding solve time is consequently dependent on which level of solver is used, ranging from milliseconds to minutes per step.

If a warning or fatal event occurs in the FMU, the documentation states it should terminate the program. This is not always the case as there are several warnings which terminate the FMU, but do not provide a python exit status. Utilizing the FMU's exit status instead guarantees the program will handle errors correctly.

5.1.3. Reference Ignition Sequence Simulation

To validate the FMU's functionality, the reference ignition sequence was tested through a separate script to ensure the LRE's internal state matched Amesim's. Past the validation, the reference ignition sequence will act as an end goal and will not be used in the training procedure.

Amesim contains the reference ignition sequence in a custom table format which was converted to a standard .csv file. The reference ignition sequence did not contain positions at standard intervals and therefore an interpolating function was required to properly access the positions at all time steps. Additionally, the reference ignition sequence columns were reversed inside of Amesim, so the program was modified to account for this.

Variable and CPU utilization were plotted to verify the results against Amesim's results. The difference between the Amesim results and FMU results were negligible aside from the total real time needed for simulation. On average, the FMU requires nearly 4x the amount of time compared to Amesim given the exact same simulation parameters. It is currently unknown why this is the case and finding a solution would significantly speed up agent training.

6. RESULTS AND ANALYSIS

6.1. Toy Problems

Due to the long training time and significant testing and validating, running the full RS-25 simulation within our time frame is unrealistic. Instead, a set of toy problems were created to test the agent's ability to learn to open the valves according to some function. These are designed to run outside of the FMU, requiring seconds per episode compared to seconds per step.

6.1.1. Linear State Actions

Starting from the basics, the simplest environment-reward structure was determined to be a linearly increasing function at the maximum valve speed. More specifically, the 30 agent states are set to 0 and increase over time at the same rate as the maximum valve speed. The agent then receives a maximal reward for opening the valves at their maximum speed.

6.1.1.1. Static Entropy and LR

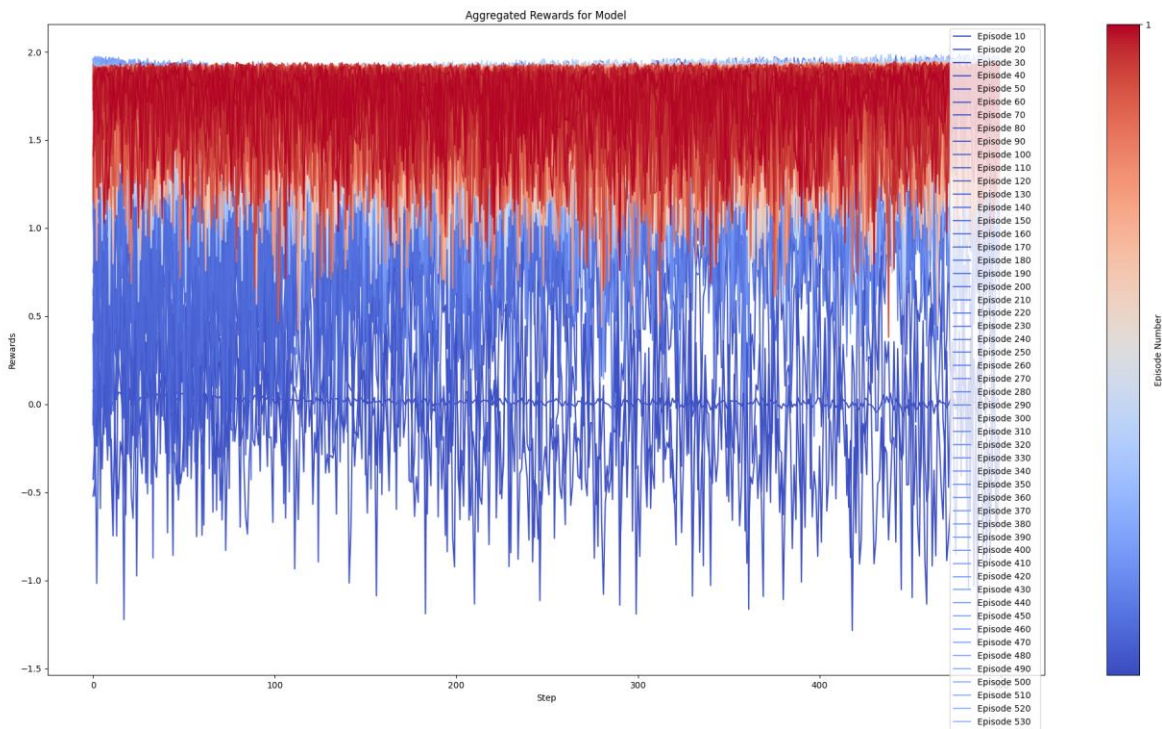


Figure 6-1.1. static entropy and LR reward history.

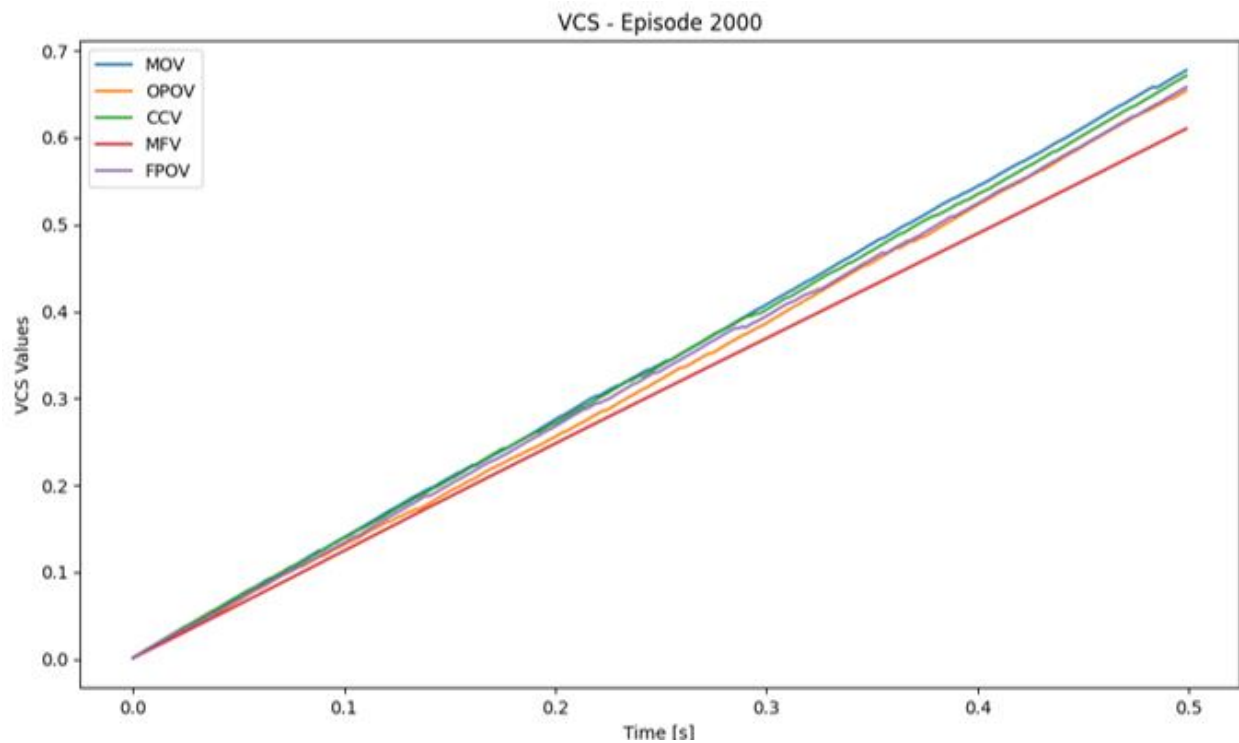


Figure 6-1.2. static entropy and LR final ignition sequence.

Figure 6-1.1 displays the aggregated rewards for each episode from the first episode (in dark blue) to the last episode (in dark red). Figure 6-1.2 displays the plot for the final episode's "ignition sequence".

The reward maximum is 2.0 and the average episode reward after training is 1.6 with drastic changes in the reward at each time step within an episode. This likely indicates that both the most optimal policy has not been found and the actor's behavior is not stable. Part of the solution was to add in the dynamic entropy term in order to reduce the stochasticity of the actor. This the training and behavior of this term is described in section 3.2.3 and section 3.4.1 respectively.

6.1.1.2. Dynamic Entropy

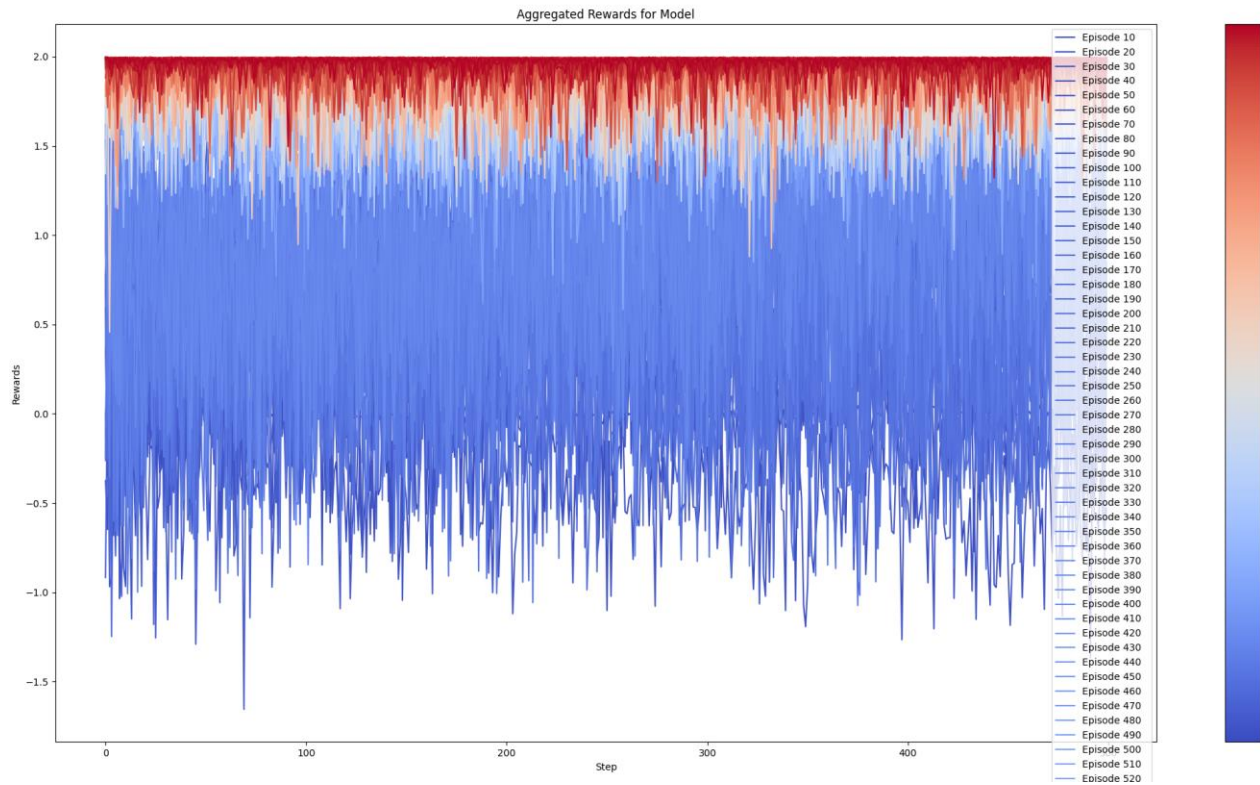


Figure 6-2.1. dynamic entropy reward history.

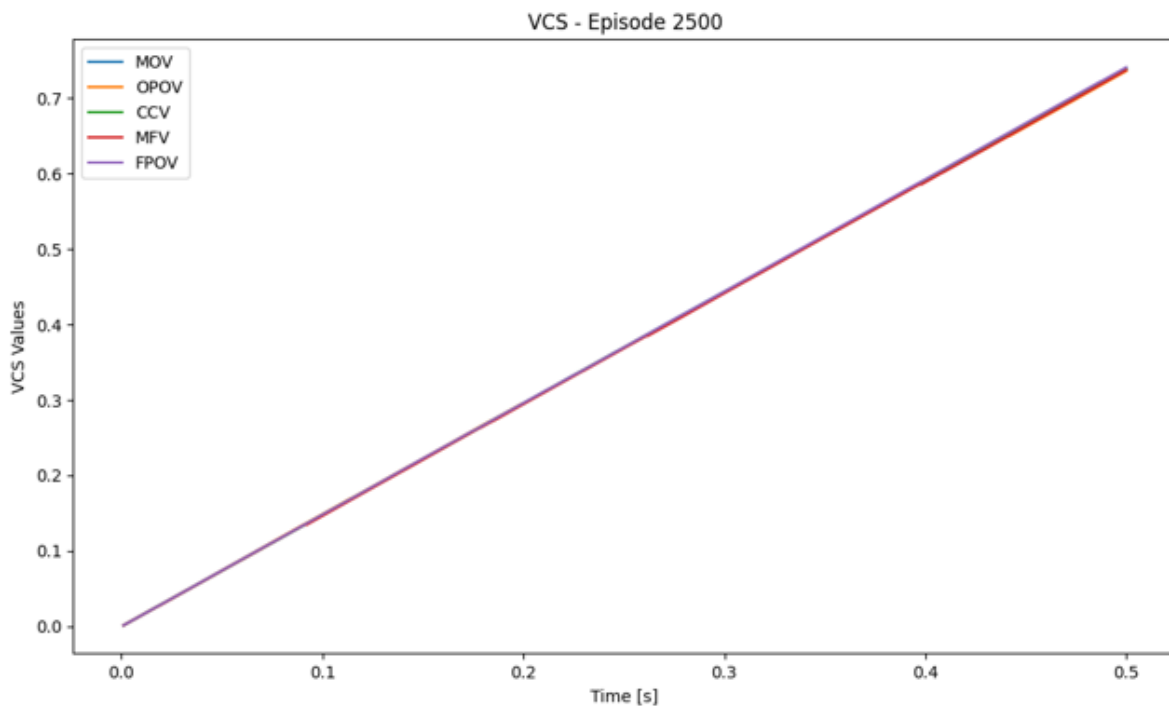


Figure 6-2.2. dynamic entropy final ignition sequence.

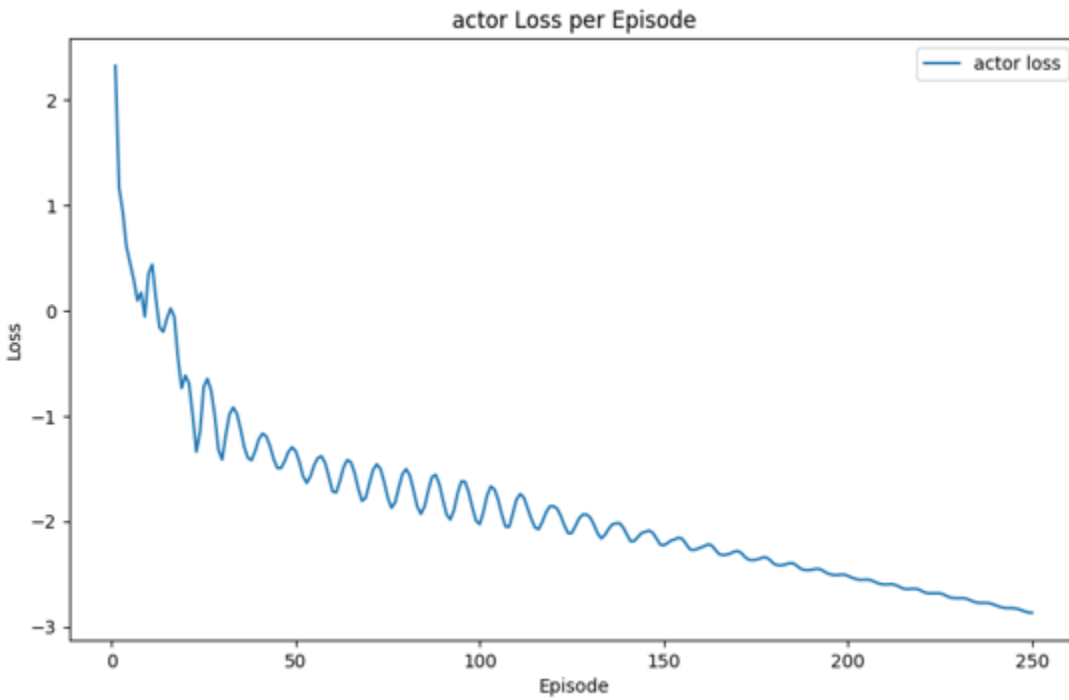


Figure 6-2.3. dynamic entropy actor loss.

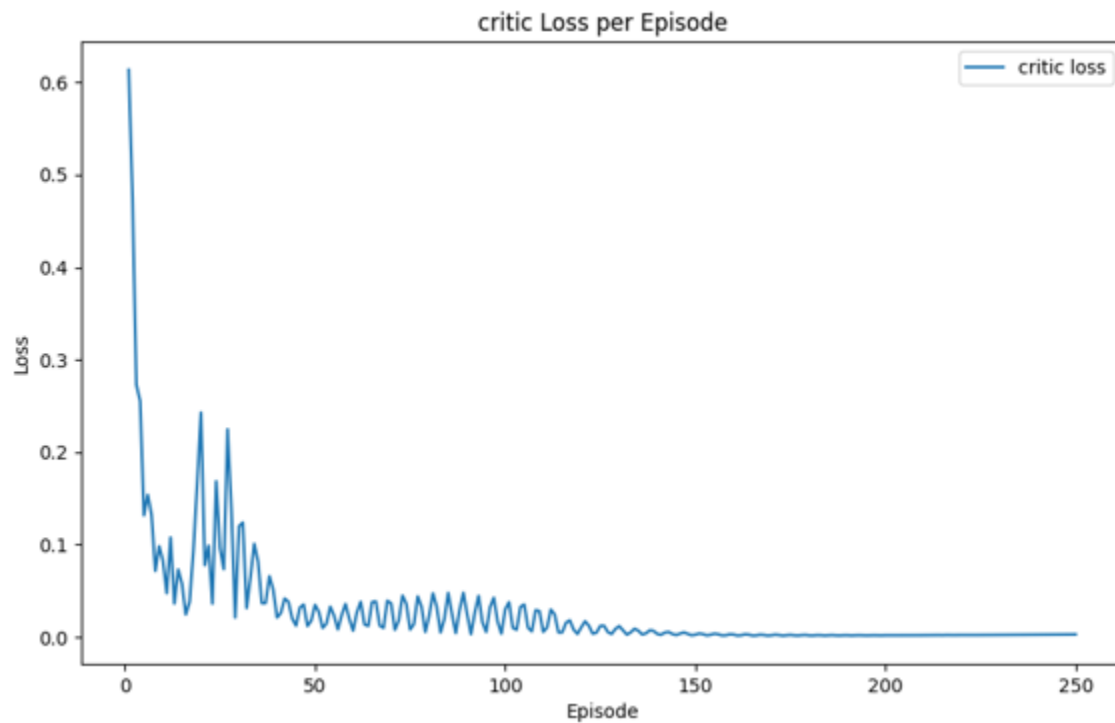


Figure 6-2.4. dynamic entropy critic loss.

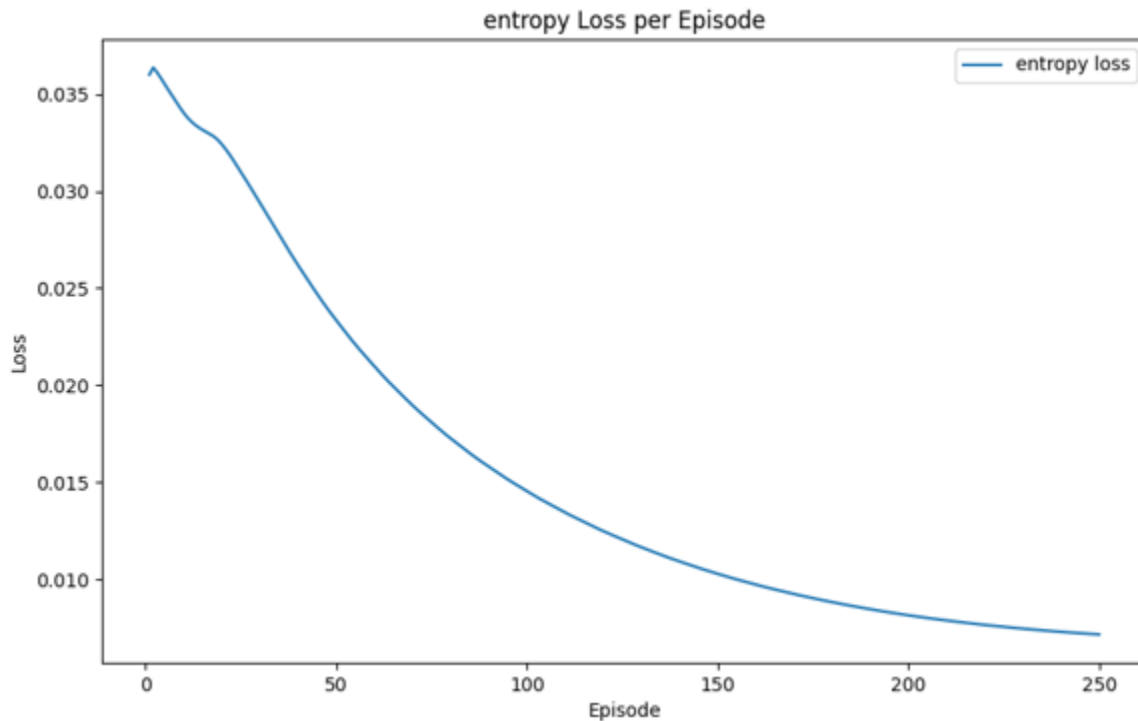


Figure 6-2.5. dynamic entropy entropy term.

Adding in the dynamic entropy term increased the mean reward from 1.6 to 1.9 out of 2.0 which is a significant improvement as seen in Figure 6-2.1. Additionally, the variability in the reward at each time step has decreased significantly which can be seen evidently through near-perfect match of the intended slope in Figure 6-2.2.

Figures 6-2.3 through 6-2.5 plot the losses associated with the trainable parameters of the agent. This run was done after manipulating the agent's hyperparameters to promote finding the optimal policy nearly instantly. Therefore, all three graphs converge towards their local minimums in under 150 episodes. Training beyond this point merely reinforced the current policy with insignificant overall gains.

A noteworthy aspect of the actor and critic losses was the oscillatory behavior before policy convergence. This is likely attributed a relatively large LR ($1e-2$) which causes the parameters to overshoot their local maximums. A physical example would be standing near the top of a hill and taking a step results in being on the opposite side of the hill. Unless smaller steps are taken, there's no way to reach the top of the hill. Therefore, a further refinement was added where the LR would decrease if progress towards the maximum halted using the [ReduceLROnPlateau](#) function from PyTorch.

6.1.1.3. Dynamic Entropy and LR

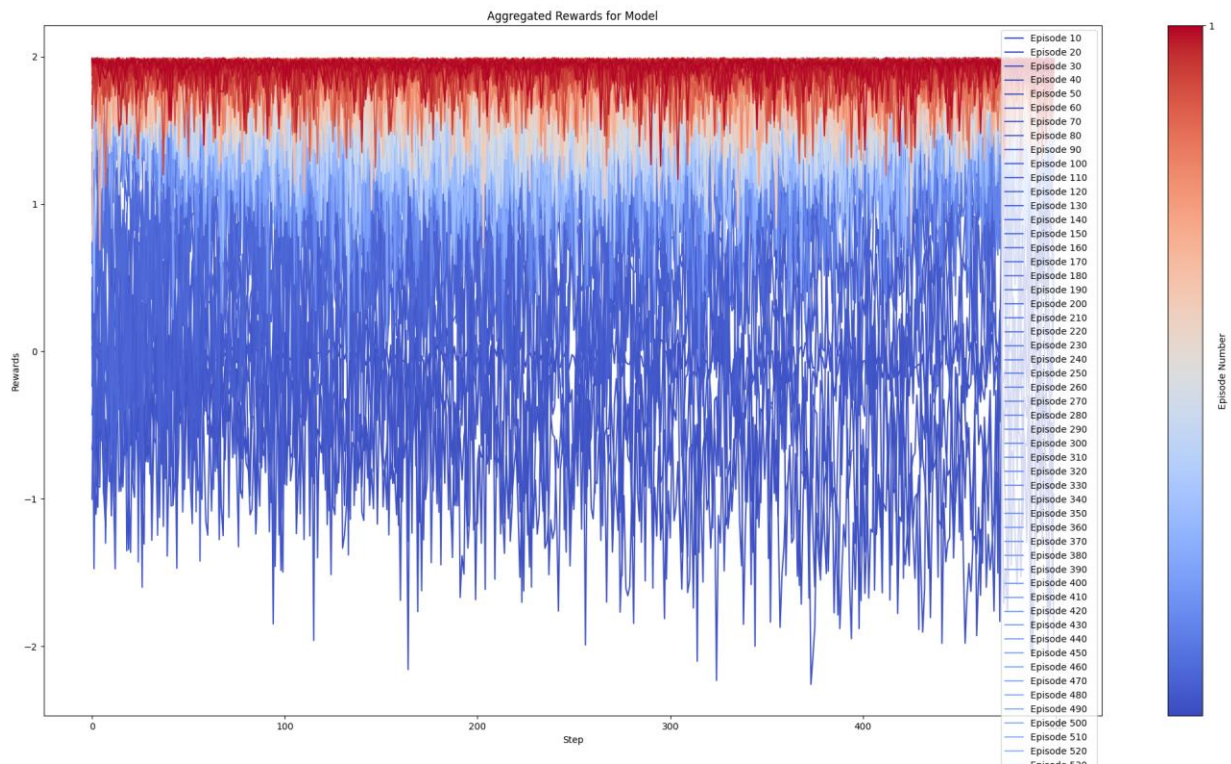


Figure 6-3.1. dynamic entropy and LR reward history.

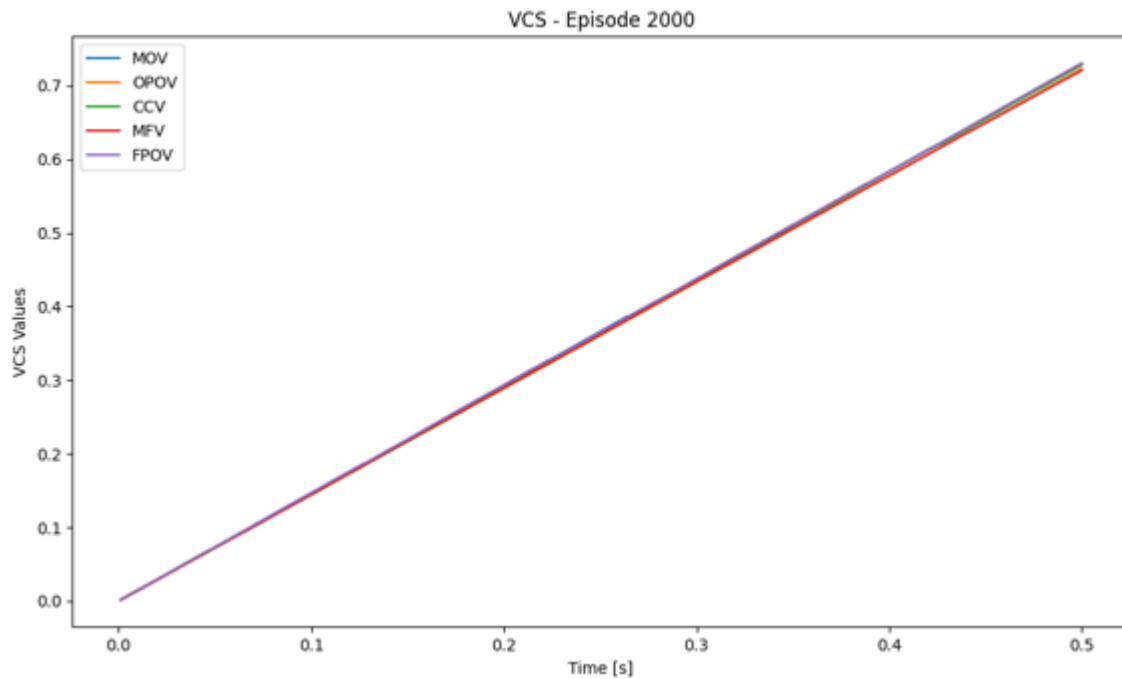


Figure 6-3.2. dynamic entropy and LR final ignition sequence.

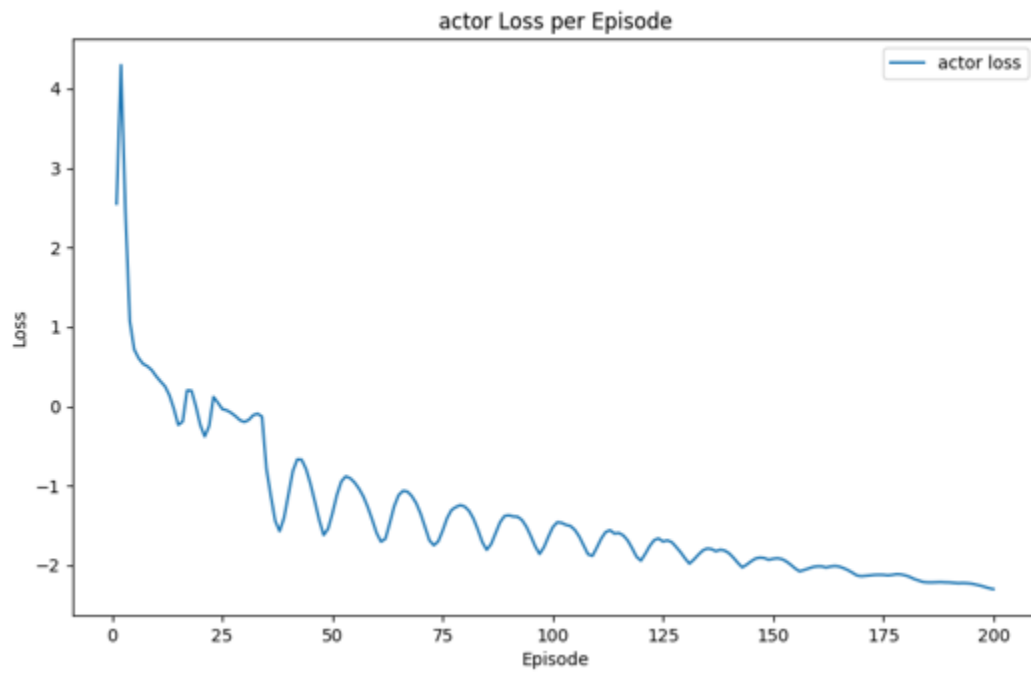


Figure 6-3.3. dynamic entropy and LR actor loss.

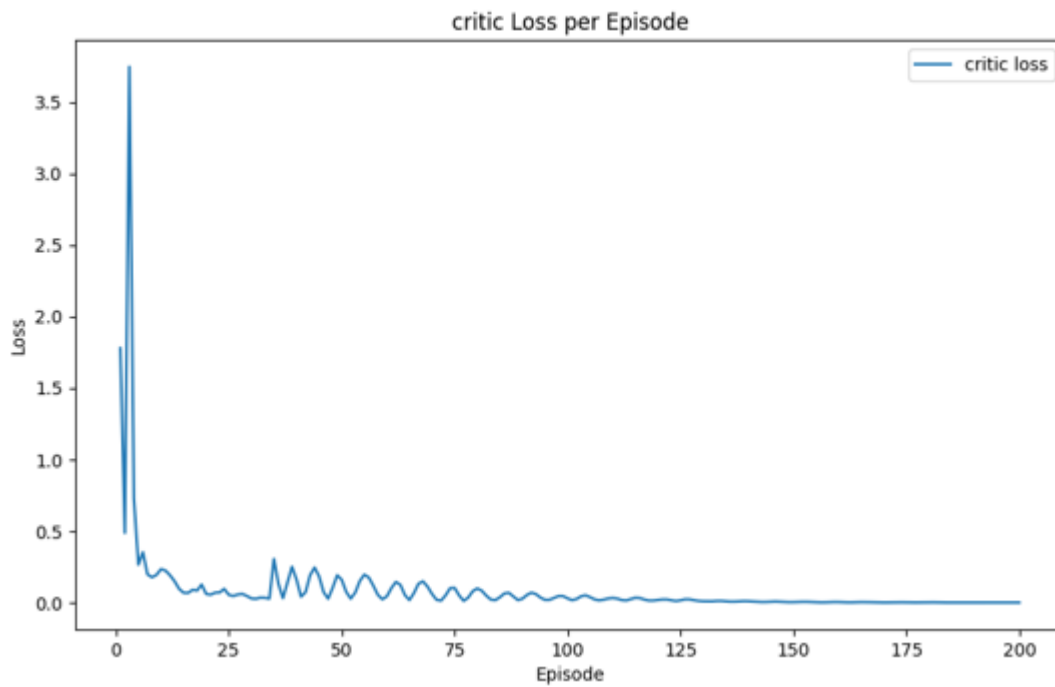


Figure 6-3.4. dynamic entropy and LR critic loss.

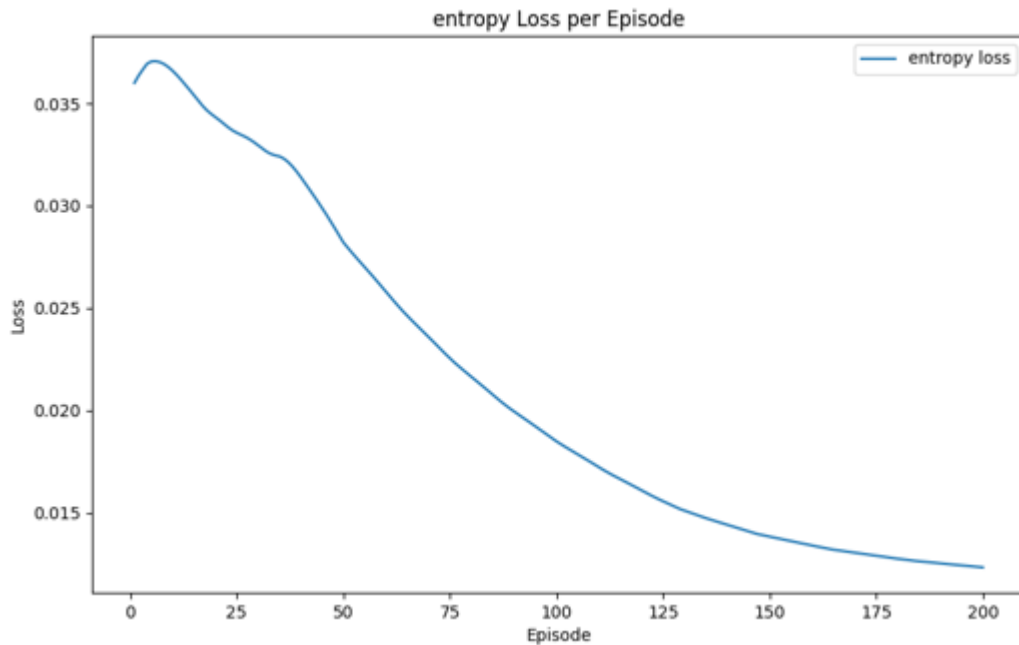


Figure 6-3.5. dynamic entropy and LR entropy term.

The average reward increased from 1.9 to 1.95 out of 2.0 and the agent learned the optimal policy faster than with a static LR, converging in under 50 episodes. The drastic improvement is likely partly due to better parameters randomization during initialization compared to purely from the dynamic LR. However, the dynamic LR did improve the training stability during the oscillatory period as there was no average increase in either actor or critic losses compared to without a dynamic LR.

The progression criteria chosen for the displayed figures was the average reward for each episode. This proved to work well for fine-tuned hyperparameters, but other testing indicated that poorly-tuned models never converged utilizing the rewards as progression criteria. This is primarily due to the specifics of how the ReduceLROnPlateau function works as oscillatory behavior mixed with exploratory behavior is difficult to account for.

More realistic progression criteria, such as a scaled linear combination of the loss functions, proved to work in general cases, however significant tuning was required for only marginal improvement. The most realistic use case of the dynamic LR function is after the model has been coarsely tuned and the path towards the optimal policy has been found. Currently, this would be a manual task, however it is likely an automatable task.

7. FUTURE WORK

This section outlines the required work to bring the project to a completed state by finishing the goals outlined in section 1.2. Each subsection will include the authors' opinions on how to implement the future work which may or may not be the best path forward. It is encouraged that readers solve these issues with their own research rather than blind faith.

7.1. LRE Stability Check

Currently, the reward encoder has a function to check the system stability. However, the function is underdeveloped and neglects many parameters that affect engine stability. Research has been done on engine stability, and is currently held in the project folder [11], but have not been implemented due to difficulties finding limit values for the RS-25. More research and testing may be needed to ensure the ignition sequence provided by the agent meets design parameters and functional criteria.

7.2. Updated Reward Structure

The current reward structure works under the assumption that there are two distinct phases in the ignition sequence which require separate agents to learn all of the valve positions. This does not match the analysis in sections 2.2.2 and 2.2.3 which was determined after the reward structure was set in place. The goal is to update the reward structure to exploit the new information.

It is believed the best path would be to create two separate agents, with one functioning outside of an episode and one functioning inside an episode.

The first agent would work by outputting three parameters for each valve: start time, slope, and stop position. The second agent would work as current described in this document where it determines the valve positions, but only for the variable valves (FPOV and OPOV). During each episode, the fixed valves would directly follow the output parameters while the variable valves would loosely follow their parameters while combining input from the second agent.

Training would need to be determined as qualifying a single episode as an experience would likely increase training times rather than reducing it, but this can be solved with section 7.4. Additionally, if whole episodes are used, determining how to encode the entire episode into a reasonably compressed form is crucial. Finally, this approach may prove that other machine learning approaches are better suited for part or all of this project. The genetic algorithm is not currently used as the state space is too large (approximately a googol), however these simplifications will significantly reduce the state space.

Separately, there's a need to adjust the current rewards to be easier for the agent to learn and understand. Part of this includes modifying the violations and other harsh thresholds to gradually increase in penalty over time after starting from a lower initial value [15]. This is a very common problem with the toy problems that did not make it into section 6.1.

7.3. Clustered Environment Simulation

Currently, the FMU is single-threaded simulation and takes significantly longer than the agents to run. Therefore, finding ways to multi-thread it would speed up agent training by a large margin. Furthermore, the agent's current and updated reward structure require multiple episodes before enough experiences are gathered.

The idea is to create a network of machines running simulations which get combined in a master program. This would update the agents on all of the slave machines which would then begin their simulation again. The maximal scenario is needing each slave to only simulate a single episode and having enough slave machines to gather the required data in a single shot.

This would be large undertaking, requiring good programming skills and fairly decent IT experience, however it would greatly decrease the training time from months to hopefully under a week.

The major problem is networking the machines together and creating a shared interface for data transfers. There are likely many smaller issues with this approach which will require V&V of each subsystem to ensure functionality.

Amesim licensing with FMUs may also need to be determined before this is implemented on a large scale. Theoretically, there are only 2 Amesim licenses and each FMU requires access to one before it can run. However, the authors were able to run 3 simulations simultaneously. Further research into how the 100 floating tokens work and what is required by the FMU will be required. According to Scott Kidney, Elliot Haag is likely the best starting place for determining how the licenses work for Amesim.

STAR-CCM+ currently has capabilities to run on the cluster, as described on [ATAWiki](#). It may be possible to run this package on the cluster as well, which would be the ideal case.

7.4. Agent Pre-Training

Given the improvements made through sections 7.2 and 7.3, it is likely a pre-training step could be implemented to greatly decrease the training time given general behavior. Pre-training would be through a synthetic environment and matching a known curve would run significantly faster than just standard training. Determine how the synthetic environment would function and what structure the agents should target will require a significant amount of work, but could lead to a much faster convergence.

REFERENCES

- [1] Brunton, S. L., & Kutz, J. N. (2021). Chapter 11: Reinforcement Learning. In *Data Driven Science & Engineering: Machine Learning, Dynamical Systems, and Control* (pp. 500–700). essay, Cambridge University Press.
- [2] Di Matteo, F., De Rosa, M., & Onofri, M. (2011). Start-up transient simulation of a liquid rocket engine. *47th AIAA/ASME/SAE/ASEE Joint Propulsion Conference & Exhibit*.
<https://doi.org/10.2514/6.2011-6032>
- [3] Jimenez Mena, D., Pluchart, S., Mouvand, S., & Broca, O. (2019a). Rocket Engine Digital Twin – Modeling and Simulation Benefits. *AIAA Propulsion and Energy 2019 Forum*.
<https://doi.org/10.2514/6.2019-4114>
- [4] Liu, S. (2024). An evaluation of DDPG, TD3, SAC, and PPO: Deep Reinforcement Learning Algorithms for controlling continuous system. *Proceedings of the 2023 International Conference on Data Science, Advanced Algorithm and Intelligent Computing (DAI 2023)*, 15–24.
https://doi.org/10.2991/978-94-6463-370-2_3
- [5] Maddali, S. (2022, May 12). *What are the most important preprocessing steps in machine learning and Data Science?*. Medium. <https://towardsdatascience.com/what-are-the-most-important-preprocessing-steps-in-machine-learning-and-data-science-a7606d18f32a>
- [6] Mill, T. R., & Breen, B. P. (1970). Computer Modeling of Rocket Engine Ignition Transients.
- [7] NASA. (1974). *Turbopump systems for liquid rocket engines*.
- [8] Park, S.-Y., & Ahn, J. (2020). Deep neural network approach for fault detection and diagnosis during startup transient of liquid-propellant rocket engine. *Acta Astronautica*, 177, 714–730.
<https://doi.org/10.1016/j.actaastro.2020.08.019>
- [9] Park, S.-Y., Kim, Y., & Jeong, E.-W. (2017). Optimization of the Startup Sequence of a Liquid-propellant Rocket Engine. *7TH EUROPEAN CONFERENCE FOR AERONAUTICS AND SPACE SCIENCES (EUCASS)*. <https://doi.org/0.13009/EUCASS2017-293>
- [10] Sesnic, T. (2023, March 10). *How to start a rocket engine*. Everyday Astronaut.
<https://everydayastronaut.com/how-to-start-a-rocket-engine/>
- [11] Seymour, D. C., Martin, M. A., Nguyen, H. H., & Greene, W. D. (2003, October 24-27). *Transient Mathematical Modeling for Liquid Rocket Engine Systems: Methods, Capabilities, and Experience*. Fifth International Symposium on Space Propulsion. Chattanooga, Tennessee, USA.
- [12] Sutton, G. P., & Biblarz, O. (2017). *Rocket Propulsion Elements* (9th ed.). Wiley.
- [13] Vetcha, N., Strickland, M. B., Philippart, K. D., & Giel, T. V. (2018). Overview of RS-25 adaptation Hot-Fire Test series for SLS, status and lessons learned. *2018 Joint Propulsion Conference*. <https://doi.org/10.2514/6.2018-4459>
- [14] Waxenegger-Wilfing, G., Dresia, K., Deeken, J., & Oschwald, M. (2021). A reinforcement learning approach for transient control of liquid rocket engines. *IEEE Transactions on Aerospace and Electronic Systems*, 57(5), 2938–2952. <https://doi.org/10.1109/taes.2021.3074134>

- [15] Yoo, H., Zavala, V. M., & Lee, J. H. (2021). A dynamic penalty function approach for constraint-handling in reinforcement learning. *IFAC-PapersOnLine*, 54(3), 487–491.
<https://doi.org/10.1016/j.ifacol.2021.08.289>

Appendix A Training Materials

A.1 Introductory Material

[3Blue1Brown's introductory course over neural network basics](#) (watch the full video series)

[Introduction to Tensors](#) (not required, but encouraged)

[Python Basics](#) (not required, but encouraged)

[Full PyTorch Tutorial](#) (not required, but encouraged)

A.2 Primary Material

[Steven Brunton's Reinforcement Learning lecture series](#) (watch only videos over policy-free, gradient based architectures)

[Soft Actor-Critic python tutorial](#)

A.3 Miscellaneous

[Steven Brunton's Data Driven Science textbook](#) (not required, but encouraged)

Appendix B Additional Sources

- [15] Blevins, G. (2023). *SpaceX's next-generation Starship spacecraft, atop its powerful Super Heavy rocket, falls back to earth as it explodes following the launch from the company's Boca Chica launchpad on a brief uncrewed test flight.* photograph, Brownsville, Texas.
- [16] Boeing. (1998). *SSME Start Sequence*.
- [17] *Functional Mock-up Interface Specification*. Functional Mock-Up Interface Standard. (2008). <https://fmi-standard.org/docs/3.0.1/>
- [18] Kadari, P. (2024, February 21). *What is reinforcement learning and how does it work (updated 2024)*. Analytics Vidhya. <https://www.analyticsvidhya.com/blog/2021/02/introduction-to-reinforcement-learning-for-beginners/>
- [19] Kim, Byeongjun & Kwon, Gunam & Park, Chaneun & Kwon, Nam. (2023). The Task Decomposition and Dedicated Reward-System-Based Reinforcement Learning Algorithm for Pick-and-Place. *Biomimetics*. 8. 240. 10.3390/biomimetics8020240.
- [20] *Simcenter Amesim simulation model*. (2022). Femto Engineering. Retrieved from <https://www.femto.eu/stories/simcenter-amesim-sea-routing-and-speed-comparison/>.